

PROJECT REPORT

ASCII Video Player

A real-time terminal video player in C++17 that converts every frame into coloured ASCII text



Project	asciplayer: terminal RGB ASCII video player
Technologies	C++17 · FFmpeg · POSIX · ANSI true-colour terminals
Submitted by	[Your name]
Date	18 September 2026

Table of Contents

Abstract.....	3
1. Introduction.....	4
1.1 Background.....	4
1.2 Problem Statement.....	4
1.3 Objectives.....	4
1.4 Scope.....	4
1.5 Organisation of this Report.....	5
2. Tools and Technologies.....	5
2.1 C++17.....	6
2.2 Compiler and Build System.....	6
2.3 The FFmpeg Suite.....	7
2.4 POSIX Operating-System Interfaces.....	7
2.5 ANSI / VT Escape Sequences.....	8
2.6 Terminal Emulators.....	8
2.7 AppleScript and the Menlo Font.....	8
2.8 Development and Documentation Tools.....	8
3. System Architecture.....	9
3.1 Design Philosophy.....	9
3.2 End-to-End Data Flow.....	9
3.3 Directory Structure.....	10
3.4 Module Dependency Graph.....	10
3.5 State Outside the Player.....	11
3.6 Start-up Sequence.....	11
3.7 Frame Data Format and Memory.....	11
4. Pixel-to-ASCII Conversion: Techniques and Algorithms.....	12
4.1 Overview of the Four Rendering Modes.....	12
4.2 Step 1: Scaling to the Character Grid and Preserving Aspect Ratio.....	13
4.3 Step 2: Measuring Brightness (Luminance).....	14
4.4 Step 3: The Brightness Ramp.....	14
4.5 Step 4: The Square-Root Lift and the Three Styles.....	15
4.6 Step 5: Colour Quantisation.....	17
4.7 The Half-Block Technique (RGB BLOCKS).....	18
4.8 Shape Matching (RGB SHAPES).....	18
4.9 Visual Comparison on a Real Frame.....	21
4.10 Algorithm Complexity.....	23
5. Rendering Methods.....	24
5.1 The Terminal as a Display Device.....	24
5.2 Building a Frame as One String.....	24
5.3 Synchronized Updates.....	25
5.4 The Three Render Loops.....	25
5.5 Delta Rendering: Redraw Only What Changed.....	25
5.6 Colour Reuse.....	25
5.7 The Byte Budget and Adaptive Colour Tolerance.....	25
5.8 Status Bar.....	26
6. Module Descriptions.....	27
6.1 core/common: The Clock.....	27
6.2 terminal/terminal: Taking Over the Terminal.....	28
6.3 terminal/input: Reading Keys.....	29
6.4 ansi.h: Escape-Code Builders.....	29
6.5 platform/shell: Running Other Programs.....	30
6.6 platform/zoom: Smaller Characters on macOS.....	30
6.7 media/probe: What Is in the File.....	31
6.8 media/audio: The Sound Pipeline.....	31
6.9 render/modes: Modes, Ramps and Tuning.....	32
6.10 render/glyphs: The Ink Table.....	32
6.11 render/shape_lut: The Shape Matcher.....	32
6.12 player/player: The Main Loop.....	33
6.13 ui/browser: The File Picker.....	34
6.14 ui/message: Messages and Errors.....	34
6.15 main.cpp: Start-up.....	34

7. Playback Engine: Timing, Pacing and Synchronisation.....	35
7.1 The Main Loop.....	35
7.2 One Wait for Both Keys and Frames.....	36
7.3 Position from the Frame Count.....	36
7.4 Audio/Video Synchronisation.....	36
7.5 Frame Skipping.....	37
7.6 Events that Restart Playback.....	38
8. Process and Resource Management.....	38
8.1 The Video Decoder.....	39
8.2 The Audio Process Group.....	39
8.3 The Watchdog.....	39
8.4 Closing Inherited Descriptors.....	39
8.5 Signals.....	40
8.6 Always Restoring the Terminal.....	40
9. User Interface and Controls.....	40
9.1 Playback Controls.....	40
9.2 Command Line and Environment.....	40
9.3 Screens.....	41
9.4 Status Bar Behaviour.....	41
10. Performance Evaluation.....	41
10.1 Test Environment.....	41
10.2 Bytes per Frame.....	41
10.3 Terminal CPU Load and Delivered Frame Rate.....	42
10.4 Cost of Shape Matching.....	42
10.5 Memory Use.....	43
11. Build, Run and Testing.....	43
11.1 Build Process.....	43
11.2 Test Media.....	44
11.3 Verification Performed.....	44
12. Challenges and Solutions.....	44
12.1 Case Study: The Stutter Investigation.....	45
12.2 Case Study: The Refactor.....	45
13. Limitations and Future Work.....	45
13.1 Current Limitations.....	45
13.2 Future Work.....	46
14. Conclusion.....	46
References.....	46
Appendix A. Quick Reference.....	47
Appendix B. Glyph Ink Table.....	48
Appendix C. ANSI Escape Sequences Used.....	48
Keyboard input sequences decoded.....	49
Appendix D. Glossary.....	49

Abstract

This report documents **ascioplayer**, a video player written in C++17 that plays ordinary video files (MP4, MKV, MOV, WebM and others) inside a text terminal. It opens no graphics window and uses no GPU API or image library. Every frame of the film is turned into characters and 24-bit colour escape codes and written to standard output, while the soundtrack plays in step through a separate audio pipeline.

The project splits the work in two. Decoding video is a solved problem, so it is handed entirely to FFmpeg: **ffprobe** reads the file's metadata, **ffmpeg** decodes and resamples each frame to exactly the size of the terminal's character grid, and **ffplay** plays the sound. The part the project owns is turning pixels into text. It implements four rendering techniques: a luminance-to-character **brightness ramp** (in colour and monochrome), a **2 × 4 sub-pixel shape matcher** that picks the glyph whose measured ink distribution best fits each cell, and a **half-block** mode that draws two square pixels per character cell. Each has tuned variants. A square-root "lift" keeps dark scenes readable, and a 24 KB precomputed lookup table makes shape matching a single array read per cell.

Terminals cannot absorb unlimited data, so a large part of the engineering is in rendering efficiently. The renderer redraws only the cells that changed, reuses colour escape codes, treats near-identical colours as equal under an adaptive tolerance, and adjusts its per-frame byte budget from measured write times. On a 320 × 80 cell grid (25,600 cells), these methods cut the average RGB-ASCII frame from **597 KB** (full redraw) to **125 KB**, which brings it inside the 50 ms time slot of a 20 fps frame on macOS Terminal.app. Picture and sound are kept together by treating ffplay's reported audio clock as the master clock and correcting the video timeline towards it with damping.

The code base is about 2,000 lines, organised into 15 modules with documented interfaces. The report covers the tools used, the architecture, every conversion algorithm (with worked numerical examples and images rendered from real video frames), the rendering methods, each module's job and working, the timing and synchronisation engine, process management, performance measurements, testing, the problems met during development and how they were solved, and the project's current limitations.

Table 1. Project at a glance

Item	Summary
Language / standard	C++17 (Apple clang 17, g++ compatible)
Build system	GNU Make with automatic header-dependency tracking
External tools	FFmpeg 8.1 suite: ffprobe, ffmpeg, ffplay
Platform	POSIX terminals (macOS, Linux); macOS Terminal.app and Ghostty tested
Rendering modes	RGB ASCII, MONO ASCII, RGB SHAPES, RGB BLOCKS
Character sets	classic (10 glyphs), detailed (70), letters (22); 95 measured glyphs for shapes
Frame rate	20 fps in ramp modes (source rate in the other modes)
Code size	14 source files, 14 headers, approx. 2,000 lines

1. Introduction

1.1 Background

ASCII art is the practice of building pictures out of printable characters. Denser characters such as @, # and M read as dark or bright patches, depending on the background, and sparse ones such as ., : and - read as light touches. The idea is older than bitmap displays: early line printers and teletypes were used to print portraits long before computers could show images. Modern terminal emulators add two things that change what is possible. They can display **24-bit ("true") colour** for every character, and they can redraw a full screen of text many times per second.

Together these make it possible to play video in a terminal. A 1280 × 640 film frame can be shrunk to, for example, 320 × 80 samples, and each sample becomes one coloured character. At twenty frames per second the result is recognisably the film, drawn entirely in text. The terminal becomes a very low-resolution display whose pixels happen to be letters.

1.2 Problem Statement

Doing this well is harder than it looks. The naive approach (for each pixel, move the cursor, set a colour and print a character) runs into four problems:

1. **Picture quality.** Choosing a character from a pixel's brightness alone throws away the structure inside each character cell, so edges blur. Using brightness to pick both the character and its colour makes dark scenes almost black.
2. **Bandwidth.** A single naive full-colour frame for a 25,600-cell grid is about 1.2 MB. At 20 fps that is 24 MB/s, several times more than a terminal emulator such as Terminal.app can parse and paint (about 5 MB/s). The terminal falls behind and the picture freezes.
3. **Timing and synchronisation.** Picture and sound are produced by different processes that start at different moments and run on different clocks. Without correction they drift apart, and lip-sync visibly breaks within seconds.
4. **Robustness.** A terminal program changes global state (raw mode, alternate screen, font size) and starts child processes. If it crashes or is interrupted, it must still give the terminal back in a usable state and never leave sound playing with no way to stop it.

1.3 Objectives

- Play any video file FFmpeg can decode, with sound, inside a standard terminal.
- Provide several rendering techniques and let the user switch between them while the video plays.
- Keep picture and sound synchronised to within the limit of human perception (about ±50 ms).
- Keep every frame within what the terminal can draw in one frame period, without visibly lowering quality.
- Offer media-player controls: pause, seek, mute, mode, character set, style and text size.
- Structure the code as small, documented modules that someone learning C++ and systems programming can read in order.

1.4 Scope

The player targets POSIX systems (macOS and Linux) and any terminal that understands ANSI/VT escape sequences with 24-bit colour. Automatic font zoom is implemented for macOS Terminal.app through AppleScript. A launcher option opens the player in the GPU-accelerated Ghostty terminal. Windows consoles, subtitles, playlists and network streaming protocols are outside the current scope (see Chapter 13).

1.5 Organisation of this Report

Table 2. Structure of the report

Chapter	Contents
2 Tools and Technologies	Every tool, library, API and standard used to build and run the project
3 System Architecture	Data flow, directory layout, module dependencies, start-up sequence
4 Pixel-to-ASCII Conversion	Scaling, luminance, brightness ramps, lift, colour, half-blocks, shape matching
5 Rendering Methods	ANSI output, frame assembly, delta rendering, byte budget, adaptive tolerance
6 Module Descriptions	Purpose, job, interface, working and example for each of the 15 modules
7 Playback Engine	Main loop, frame pacing, audio/video synchronisation, frame skipping
8 Process and Resource Management	Child processes, pipes, process groups, signals, clean-up
9 User Interface	Controls, file browser, status bar, error screens, Ghostty launcher
10 Performance Evaluation	Measured bytes per frame, write time, CPU load, lookup-table cost
11 Build, Run and Testing	Build process, commands, verification performed
12 Challenges and Solutions	Problems met during development and how each was solved
13 Limitations and Future Work	Known limitations and possible extensions
14 Conclusion	Summary of outcomes
Appendices	Key reference, full glyph ink table, ANSI reference, glossary

2. Tools and Technologies

This chapter lists everything the project uses, grouped by role. Table 3 summarises them; the sections that follow explain what each one does in the project and why it was chosen.

Table 3. Tools and technologies used

Tool / technology	Category	Version used	Role in the project
C++17	Language	ISO/IEC 14882:2017	All player code
Apple clang / g++	Compiler	Apple clang 17.0.0	Compiles src/ to object files and links the binary
GNU Make	Build system	3.81	Incremental builds with header-dependency tracking
ffprobe	Media analysis	FFmpeg 8.1	Reads width, height, frame rate, duration, audio tracks
ffmpeg	Decoder / scaler	FFmpeg 8.1	Decodes and scales video to raw RGB24; decodes audio to PCM
ffplay	Audio output	FFmpeg 8.1	Plays PCM audio and reports its playback clock
POSIX APIs	Operating system	macOS 26.5 / Linux	termios, ioctl, poll, fork, exec, pipe, signals
ANSI / VT escape codes	Display protocol	ECMA-48 + xterm extensions	Cursor motion, 24-bit colour, alternate screen
Terminal.app	Terminal emulator	2.15	Primary test terminal (CPU renderer)
Ghostty	Terminal emulator	1.3.1	GPU-accelerated terminal, launched with -g

Tool / technology	Category	Version used	Role in the project
AppleScript (osascript)	Automation	macOS built-in	Changes Terminal.app font size during playback
Menlo / Menlo-Bold	Font	macOS built-in	Playback font; source of the glyph ink measurements
Visual Studio Code	Editor / IDE	current	Editing, navigation, integrated terminal
Python 3 + matplotlib	Report tooling	3.9 / 3.9.4	Re-implementation used to produce this report's figures

2.1 C++17

C++ was chosen for predictable performance in the rendering loop, which runs hundreds of thousands of times a second, and for direct access to the POSIX system calls the player depends on. The C++17 standard specifically provides several features the code uses:

Table 4. C++17 features used

Feature	Header	Where it is used and why
<code>std::filesystem</code>	<code><filesystem></code>	File browser directory listing, absolute/canonical paths, temp-file location for ffmpeg's error log
<code>std::chrono::steady_clock</code>	<code><chrono></code>	All playback timing. A steady clock never jumps when the system time changes, unlike <code>system_clock</code>
<code>std::to_chars</code>	<code><charconv></code>	Formatting numbers into escape codes with no allocation, locale or format parsing (hot path)
<code>std::clamp</code>	<code><algorithm></code>	Bounding indices, sizes, tolerances and timeouts
<code>enum class</code>	language	Mode, Style and K (key) are strongly typed and cannot be mixed with integers by accident
<code>constexpr constants</code>	language	Tuning values (<code>kRampFps</code> , <code>kFrameBudgetStart</code> , <code>kSubX</code> ...) known at compile time
Lambdas	language	Colour-tolerance comparison inside <code>render()</code> ; sort comparators in the browser
<code>inline header functions</code>	language	<code>ansi.h</code> helpers are header-only so the compiler can inline them per cell

2.2 Compiler and Build System

The project builds with any C++17 compiler. On the development machine `g++` is Apple clang 17. The Makefile collects every `.cpp` under `src/` automatically, so a new module needs no Makefile edit. The flags are:

Table 5. Compiler flags

Flag	Meaning
<code>-std=c++17</code>	Compile as C++17
<code>-O2</code>	Optimise for speed; the per-cell render loop depends on it
<code>-Wall -Wextra</code>	Enable most compiler warnings
<code>-Wno-unused-parameter</code>	Silence one noisy warning (signal handlers take an unused argument)
<code>-Iinclude</code>	Headers are included as "asciplayer/xxx.h"
<code>-MMD -MP</code>	Write a .d file per object listing the headers it used; editing a header rebuilds only its users

2.3 The FFmpeg Suite

FFmpeg is the most widely used open-source multimedia framework. The project uses three of its command-line programs as child processes, communicating through pipes, instead of linking the libav* libraries. This keeps the build dependency-free (only the programs must be on the PATH) and lets each stage be killed and restarted independently.

ffprobe: reading metadata

```
ffprobe -v error -select_streams v:0 \
  -show_entries stream=width,height,r_frame_rate:format=duration \
  -of default=noprint_wrappers=1 'movie.mkv'
# width=1280
# height=640
# r_frame_rate=24/1
# duration=1854.763000
```

Listing 1. The ffprobe query issued by probe(), with its output for the test film.

ffmpeg: decoding and scaling video

```
ffmpeg -nostdin -loglevel error -ss 412.350 -i 'movie.mkv' \
  -an -sn -vf fps=20.00000:start_time=0,scale=320:80 \
  -f rawvideo -pix_fmt rgb24 - </dev/null 2>/tmp/ascioplayer-<pid>.log
```

Listing 2. The video decoder command built by Player::openAt().

Table 6. Video decoder options

Option	Purpose
-ss T (before -i)	Input seek: decode forward to exactly time T
-an -sn	Drop audio and subtitle streams from this process
fps=20:start_time=0	Emit a fixed 20 frames per second; pad with the first picture if the video track starts late
scale=W:H[:flags=area]	Resample to exactly the character grid. The area filter averages pixels (used for Shapes and Blocks)
-f rawvideo -pix_fmt rgb24 -	Write bare RGB bytes, 3 per pixel, row by row, to stdout

ffmpeg + ffplay: the audio pipeline

```
ffmpeg -nostdin -loglevel quiet -ss 412.350 -i 'movie.mkv' -map 0:1 -vn -sn \
  -af aresample=async=1:first_pts=0 -f s16le -ac 2 -ar 48000 - \
  | ffplay -nodisp -autoexit -loglevel quiet -stats \
  -f s16le -sample_rate 48000 -ch_layout stereo -i -
```

Listing 3. The audio pipeline started by startAudio().

ffmpeg decodes the chosen audio stream to raw signed 16-bit little-endian stereo PCM at 48 kHz. ffplay only plays it. With **-stats**, ffplay prints its playback position about every 35 ms on standard error. The player reads those lines and uses them as its master clock (Chapter 7).

2.4 POSIX Operating-System Interfaces

Table 7. POSIX interfaces used

API	Module	Use
tcgetattr / tcsetattr (termios)	terminal	Enter and leave raw mode (no echo, no line buffering)
ioctl(TIOCGWINSZ)	terminal, zoom	Terminal size in characters and, when reported, in pixels
poll()	input	Wait for a keypress with a timeout; doubles as the frame timer

API	Module	Use
<code>read()</code> / <code>write()</code>	input, terminal	Raw keyboard bytes in; frame bytes out (writeAll loops on partial writes)
<code>popen()</code> / <code>pclose()</code>	player, shell	Start ffmpeg/ffprobe/osascript and read their stdout
<code>fork()</code> / <code>execl()</code> / <code>_exit()</code>	audio	Start the audio pipeline as its own process group
<code>pipe()</code> / <code>dup2()</code> / <code>close()</code>	audio	Route ffmpeg's stderr clock to the player; detach the child from the terminal
<code>setpgid()</code> / <code>kill(-pgid)</code>	audio	Put sh, ffmpeg and ffmpeg in one group and stop them together
<code>waitpid()</code>	audio	Reap the audio group so no zombie processes remain
<code>fcntl(O_NONBLOCK, FD_CLOEXEC)</code>	audio, player	Non-blocking clock reads; stop descriptors leaking into children
<code>sigaction</code> / <code>signal</code>	terminal	SIGINT/SIGTERM -> quit flag, SIGWINCH -> resize flag, ignore SIGPIPE
<code>std::atexit</code>	main	Guarantee <code>restoreTerminal()</code> runs on every normal exit path

2.5 ANSI / VT Escape Sequences

The terminal is controlled entirely by writing text that contains escape sequences: byte `0x1B` (ESC) followed by `[` and parameters. The player uses about a dozen of them. The full list is in Appendix C, and Chapter 5 explains how they are assembled into a frame. The most important is the **true-colour SGR sequence** `ESC[38;2;R;G;Bm`, which sets the foreground to any of 16.7 million colours.

2.6 Terminal Emulators

macOS Terminal.app (version 2.15) was the main development and test terminal. It draws text on the CPU, which limits how many bytes per second it can absorb (about 5 MB/s measured), and it reports 0 for the window's pixel size. **Ghostty** (1.3.1) is a GPU-accelerated terminal that also reports pixel dimensions. The player can relaunch itself in Ghostty with `-g`, where full-screen playback uses about a quarter of the CPU (Chapter 10).

2.7 AppleScript and the Menlo Font

Picture resolution is the number of character cells, and a terminal program cannot draw a smaller character by itself. On Terminal.app, `osascript` can change the font of the running tab. During playback the player switches the tab to **5 pt Menlo-Bold** (configurable), which multiplies the number of cells, and it restores the original font afterwards. Menlo is also the font whose glyph shapes were measured for the shape matcher (Section 4.8).

2.8 Development and Documentation Tools

- **Visual Studio Code** for editing and code navigation, with its integrated terminal for building and running.
- **Claude Code** (AI coding assistant) for pair-programming during refactoring, debugging and documentation.
- **Markdown documentation** in `docs/`: a start-here guide, architecture notes, deep dives on terminals, rendering and A/V sync, and graded exercises.
- **Python 3 with NumPy, Pillow and matplotlib**, used only for this report. The rendering algorithms were re-implemented in Python from the C++ source (same constants, same glyph table, same lookup-table construction) to produce the example images, measurements and charts.

3. System Architecture

3.1 Design Philosophy

The central design decision is to **hand every hard, already-solved problem to a tool that has solved it, and keep only the part that is new.** Video decoding covers dozens of codecs and containers, hardware paths and edge cases, and FFmpeg handles all of them. By asking ffmpeg to also scale each frame to exactly the terminal's grid, the player gets high-quality resampling for free, and its input arrives in the simplest possible format: three bytes per pixel, rows top to bottom, no padding, no headers.

What remains, and what this project is about, is the conversion from those pixels to characters and colour codes, the efficient delivery of that text to the terminal, and the timing that keeps it in step with the sound.

3.2 End-to-End Data Flow

Figure 1 traces one video from file to screen. `ffprobe` describes the file; `layout()` computes the grid; two independent `ffmpeg` processes decode picture and sound from the same timestamp; the player reads raw frames, converts them to one ANSI string per frame, and writes that string to the terminal. `ffplay`'s clock feeds back into the timing.

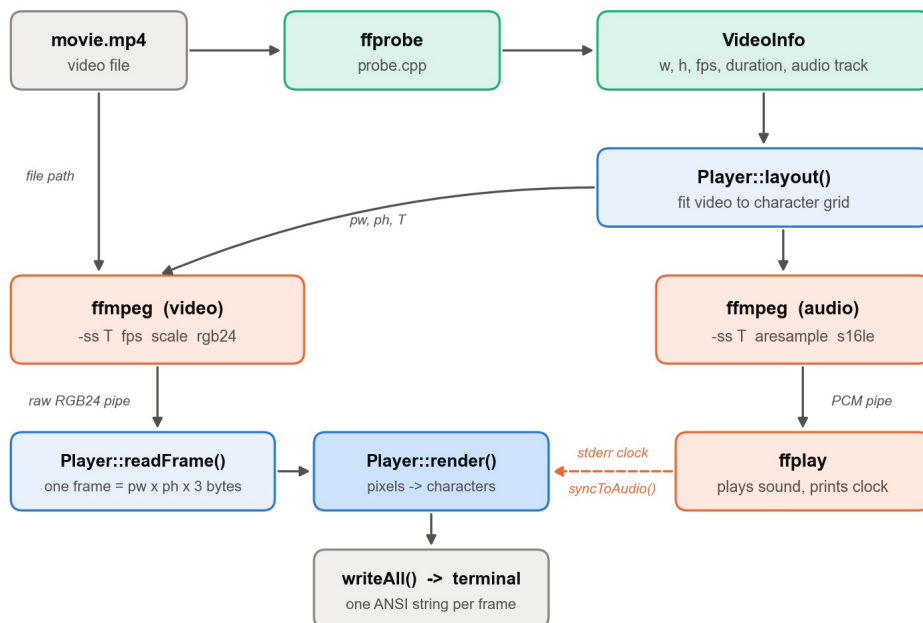


Figure 1. End-to-end data flow from video file to terminal

Two decoders are used instead of one because they are seeked, paused and restarted independently. Pausing stops the sound and keeps the video position. Seeking restarts both at an exact timestamp. Changing the render mode or resizing the window restarts only the video decoder, at a new size. The cost is that the two can drift apart, which Section 7.4 addresses.

3.3 Directory Structure

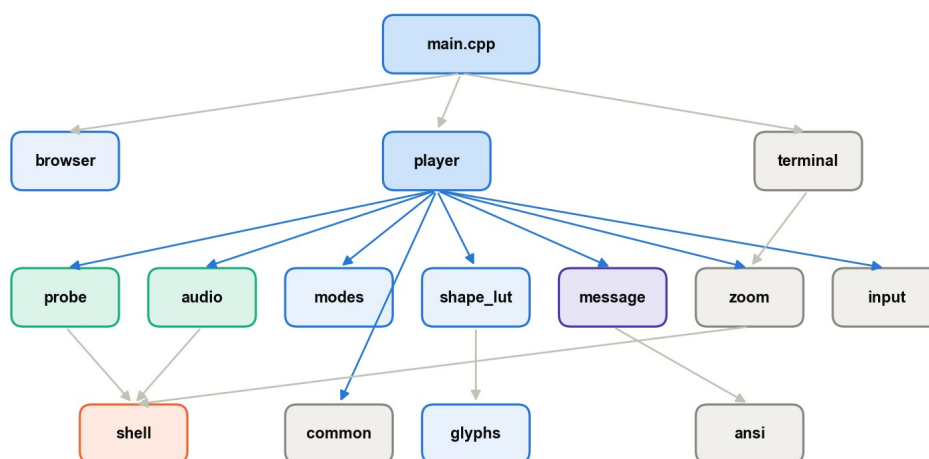


Figure 2. Project directory structure

Every module has a header in `include/asciplayer/` that explains *what* the module does and *why*, and a source file in `src/<area>/` that holds the *how*. The original single-file program (1,435 lines) is kept in `legacy/` so the modular version can be compared against it, and `make legacy` still builds it.

3.4 Module Dependency Graph

Figure 3 shows which modules include which. The graph is acyclic. `player` sits at the top and uses almost everything, and nothing but `main.cpp` depends on it. The low-level modules (`common`, `ansi`, `glyphs`, `modes`, `input`, `shell`) depend on nothing in the project and can be read and tested in isolation.



browser also uses input, ansi and terminal (edges omitted for clarity).
 Arrows point at what a module needs. Blue = render/control core, green = media, orange = process spawning, grey = terminal & platform.

Figure 3. Module dependency graph (arrows point at what a module needs)

Two dependencies deserve comment. `terminal` depends on `zoom` because `restoreTerminal()` must also undo the font zoom; otherwise quitting with Ctrl-C would leave the user's terminal at 5 pt. Because `restoreTerminal()` is registered with `std::atexit`, putting that one call there covers every exit path. And `shape_lut` depends on `glyphs` because the lookup table is computed from the glyph measurements.

3.5 State Outside the Player

Almost all state is a member of the `Player` object and is destroyed with it. Three groups of globals exist, each for a specific reason:

Table 8. Process-wide state

Global	Defined in	Why it cannot be a <code>Player</code> member
<code>g_quit</code> , <code>g_resized</code>	<code>terminal.cpp</code>	Written by a signal handler, which has no object to write to; must be volatile <code>sig_atomic_t</code>
<code>g_orig</code> , <code>g_rawOn</code>	<code>terminal.cpp</code>	The terminal's saved settings belong to the process, not to one video
<code>g_zoomOrig</code> , <code>g_zoomNow</code> , <code>g_zoomOrigFont</code> , <code>g_cellAspect</code>	<code>zoom.cpp</code>	The font belongs to the window; it must survive between videos and be restored at exit
<code>g_shapeLut</code> , <code>g_shapeLutReady</code>	<code>shape_lut.cpp</code>	Built once per process, read-only afterwards, shared by every frame

3.6 Start-up Sequence

Figure 4 shows the order of calls from launch to the first frame. `main()` checks that it is running in a terminal and that `ffmpeg` and `ffprobe` exist, installs the signal handlers and enters raw mode. The file browser returns a path. A `Player` is constructed for that file, and `run()` probes, zooms, lays out and opens the decoders before entering the main loop. When the video ends, the `Player` is destroyed (closing its pipes and killing the audio group) and control returns to the browser.

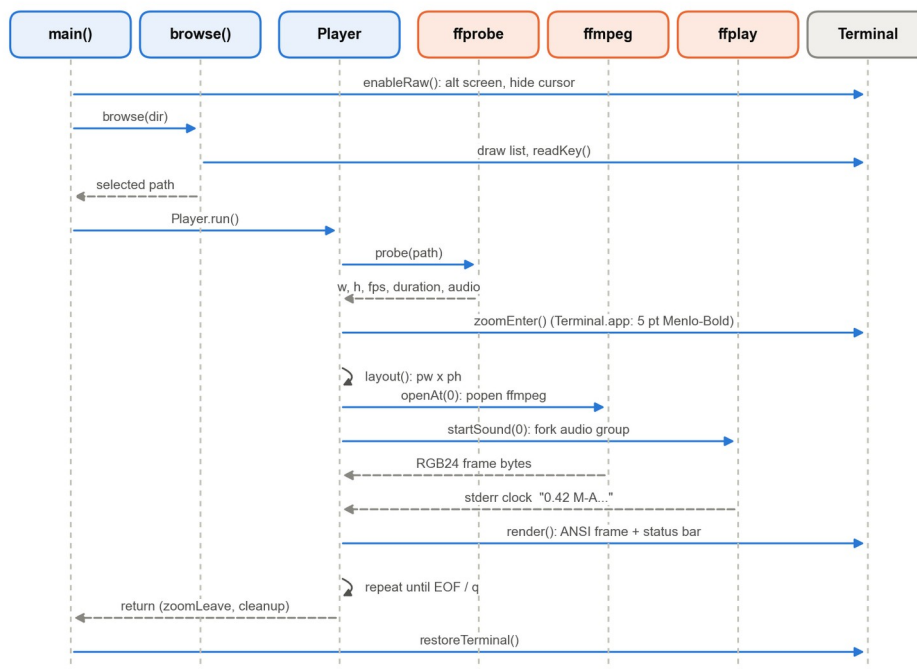


Figure 4. Start-up and playback sequence

3.7 Frame Data Format and Memory

A decoded frame is a flat `std::vector<unsigned char>` of $pw \times ph \times 3$ bytes: R, G, B for each pixel, row by row. Its size depends on the mode, because each mode asks `ffmpeg` for a different pixel grid per character cell. For a 322×90 terminal and the 1280×640 test film:

Table 9. Frame buffer sizes for a 322×90 terminal

Mode	Pixels per cell	Cells used	ffmpeg output ($pw \times ph$)	Bytes per frame
RGB / MONO ASCII	1×1	320×80	320×80	76,800

Mode	Pixels per cell	Cells used	ffmpeg output (pw × ph)	Bytes per frame
RGB SHAPES	2 × 4	322 × 81	644 × 324	625,968
RGB BLOCKS	1 × 2	322 × 80	322 × 160	154,560

Alongside the frame, ramp modes keep **drawn**, an **int** per cell recording what that cell currently shows on screen (Section 5.5), and a saved copy for retrying oversized frames. The output string **out** is reused from frame to frame, so after the first few frames rendering allocates no memory.

4. Pixel-to-ASCII Conversion: Techniques and Algorithms

This chapter is the core of the report. It explains every step between a decoded RGB frame and the character plus colour that appears in each terminal cell. Every constant quoted here comes from the source code, and every example image in this chapter was produced by running the same algorithm on real frames of the test film.

4.1 Overview of the Four Rendering Modes

The modes differ in **what one character cell represents** (Figure 5). The user cycles through them with the **m** key.

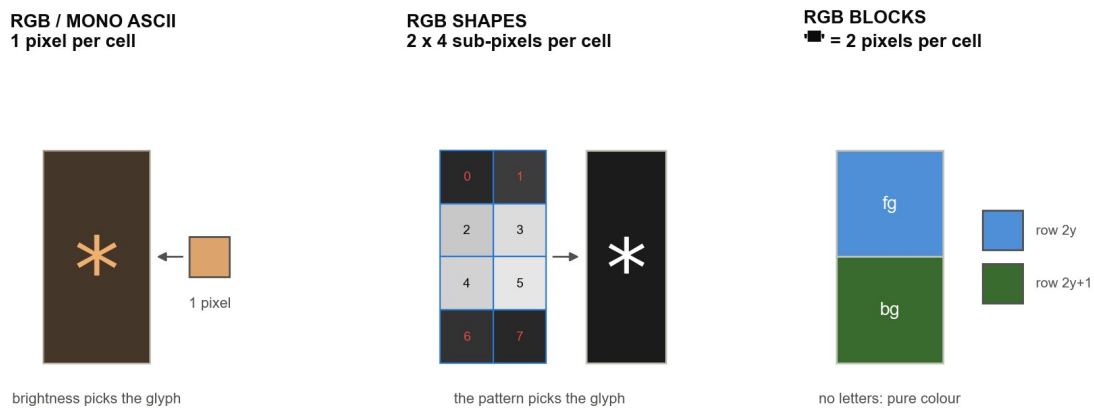


Figure 5. What one character cell represents in each mode

Table 10. The four rendering modes

Mode	Samples per cell	What picks the character	What picks the colour	Strength
RGB ASCII	1 pixel	Pixel brightness -> ramp index	Pixel colour (fg), optional half-strength bg	Classic look; the default
MONO ASCII	1 pixel	Pixel brightness -> ramp index	None (terminal default text colour)	Pure text; smallest frames
RGB SHAPES	2 × 4 sub-pixels	Best match of ink shape to the 8 samples	Average of the 8 samples	Sharpest edges from text
RGB BLOCKS	2 pixels (top, bottom)	Always the upper half block	fg = top pixel, bg = bottom pixel	Highest image fidelity

Every mode follows the same pipeline: **scale** the frame to the grid, **measure** each cell, **choose** a glyph and colours, then **emit** escape codes. The sections below take each step in turn.

4.2 Step 1: Scaling to the Character Grid and Preserving Aspect Ratio

The picture's resolution is the character grid, so the first job is to decide how many cells the picture gets and ask ffmpeg for exactly that many pixels. This is `Player::layout()`. The complication is that a character cell is not square. A typical monospace cell is about half as wide as it is tall, so a picture drawn one pixel per cell with no correction looks twice as tall as it should.

The cell aspect ratio

`cellAspect()` returns the cell's width divided by its height. When the terminal reports its size in pixels (Ghostty, kitty, iTerm2 and WezTerm fill in `ws_xpixel` and `ws_ypixel`), the value is exact:

```
cellAspect = (ws_xpixel / ws_col) / (ws_ypixel / ws_row)    clamped to [0.3, 0.7]
```

Terminal.app reports zero pixels, so there the value is measured from the window bounds and the row and column counts that AppleScript returns after the font zoom, or it falls back to 0.5.

Layout for the ramp modes

```
ratio = (video_h / video_w) * cellAspect()
w     = cols - 2                                // one-cell margin all round
h     = round(w * ratio)
if h > rows - 2: h = rows - 2; w = round(h / ratio)
pw, ph = w, h                                  // one pixel per character
offX = (cols - w) / 2; offY = (rows - 1 - h) / 2 // centre it
```

Listing 4. Ramp-mode layout (from `Player::layout()`).

Worked example: layout

Terminal: 322 columns × 90 rows. Video: 1280 × 640 (2:1). `cellAspect` = 0.5.

$\text{ratio} = 640/1280 \times 0.5 = 0.25$. $w = 322 - 2 = 320$. $h = \text{round}(320 \times 0.25) = 80$ (limit 88, so it fits).

ffmpeg is asked for `scale=320:80`: 25,600 pixels, one per cell. Offsets: $\text{offX} = (322 - 320)/2 = 1$, $\text{offY} = (89 - 80)/2 = 4$.

The SHAPES mode uses a fixed factor of 2 for the cell shape and asks ffmpeg for $2 \cdot w \times 4 \cdot h$ pixels. The BLOCKS mode asks for $w \times 2h$ pixels, forcing the pixel height even so that every cell has exactly one top and one bottom pixel. In those two modes the scale uses `flags=area`, which averages every source pixel that falls into a destination pixel. That is the right filter when shrinking by a large factor. The ramp modes use ffmpeg's default (bicubic) scaler.

Figure 6 shows the result of the scaling step for the ramp modes: the whole 1280 × 640 frame reduced to 120 × 30 pixels, one per character, enlarged here for display.

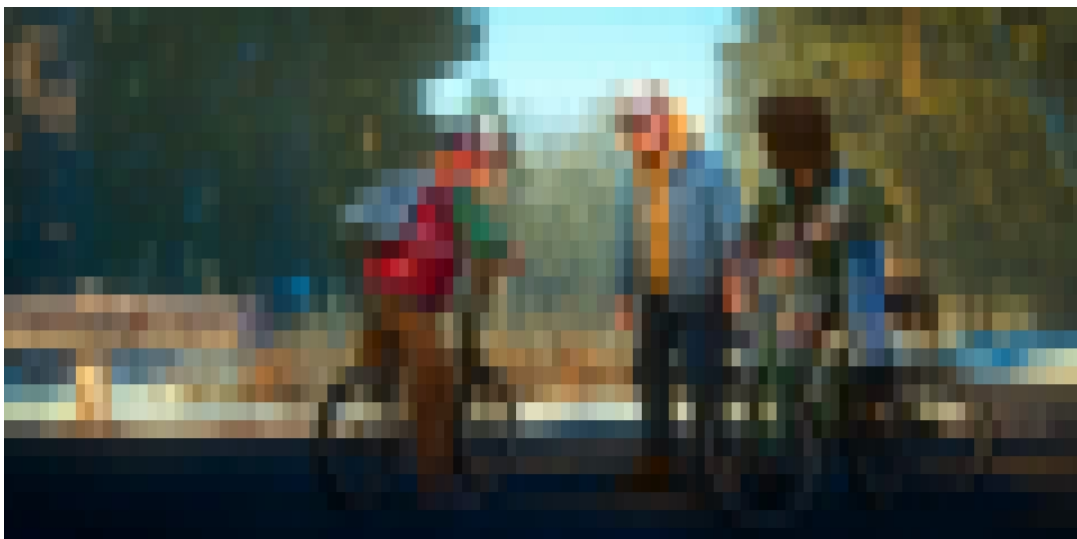


Figure 6. Step 1 output: the frame scaled by ffmpeg to 120 × 30 pixels, one per character cell (enlarged)

4.3 Step 2: Measuring Brightness (Luminance)

Every ramp decision starts from one number per pixel: its perceived brightness. The human eye is far more sensitive to green light than to red, and least sensitive to blue. A plain average $(R+G+B)/3$ would therefore make a saturated blue sky look as bright as a green field of the same channel value. The player uses the **ITU-R BT.601 luma weights**:

$$\text{lum} = 0.299 \times R + 0.587 \times G + 0.114 \times B \quad (0 \dots 255)$$

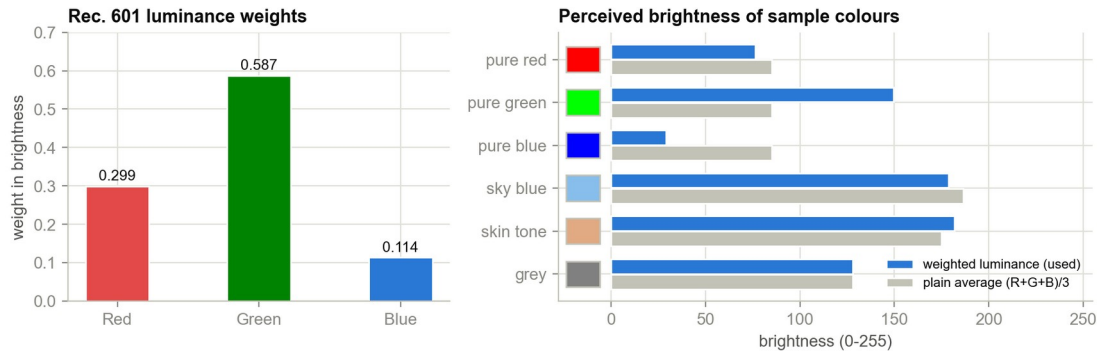


Figure 7. BT.601 luminance weights, and weighted versus plain-average brightness for sample colours

Figure 7 shows the effect. Pure blue scores 29 with weighting against 85 for a plain average, and pure green 150 against 85. In the SHAPES mode the same weights are computed in integer arithmetic, $(r \cdot 299 + g \cdot 587 + b \cdot 114) / 1000$, which avoids floating point in the hottest loop.

4.4 Step 3: The Brightness Ramp

A **ramp** is a string of characters ordered from the least ink to the most. The pixel's brightness is scaled to an index into that string:

```
top = strlen(ramp) - 1
index = clamp( int(level * top), 0, top )    // level = lum / 255 in 'exact'
glyph = ramp[index]
```

On a dark terminal background a character with more ink looks brighter, so dark pixels become spaces or dots and bright pixels become % or @. Three ramps are built in:

Table 11. Built-in character ramps

Name	Glyphs (dark -> bright)	Steps	Character
classic	" . : - = + * # % @ "	10	The familiar look; coarse but clean
detailed	" . ` ^ \ , ; ! l i > < ~ + _ - ? [] { } 1 (/ t f j r x n u v c z X Y U J C L Q 0 0 Z m w q p d b k h a o * # M W & 8 % B @ \$ "	70	Finest tonal steps
letters	" . , ; i r s X A 2 5 3 h M H G S # 9 B & @ "	22	Looks like text from a distance; never blank

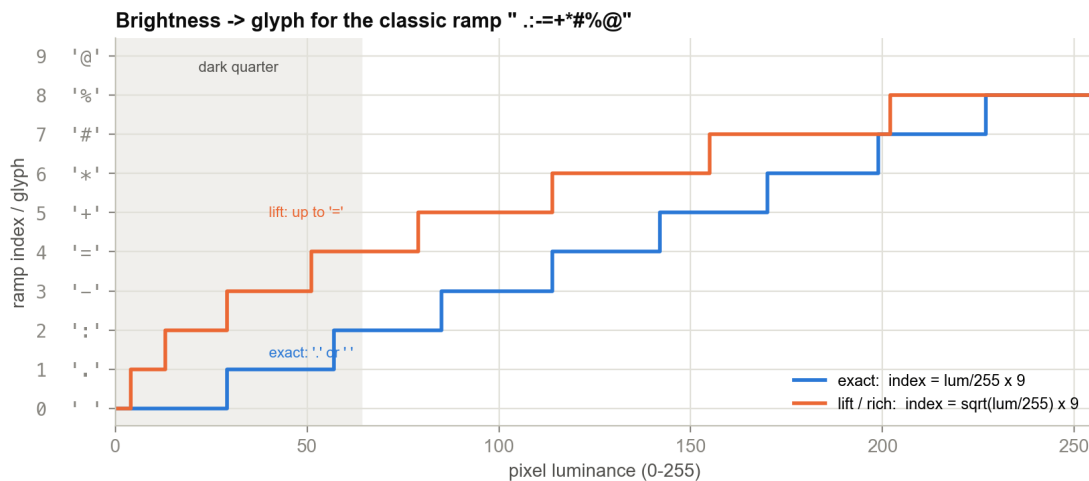


Figure 8. Mapping from pixel brightness to glyph for the classic ramp, exact versus lift

Worked example: one pixel through the ramp

Pixel RGB (220, 164, 108), a warm skin tone. $\text{lum} = 0.299 \cdot 220 + 0.587 \cdot 164 + 0.114 \cdot 108 = 65.78 + 96.27 + 12.31 = 174.36$.

exact: level = $174.36/255 = 0.6838$; index = $\text{int}(0.6838 \times 9) = 6$; glyph = ramp[6] = *.

lift: level = $\sqrt{0.6838} = 0.8269$; index = $\text{int}(0.8269 \times 9) = 7$; glyph = ramp[7] = #.

The charset changes the texture of the picture but not its tones. Figure 9 shows the same frame drawn with the detailed and letters ramps.

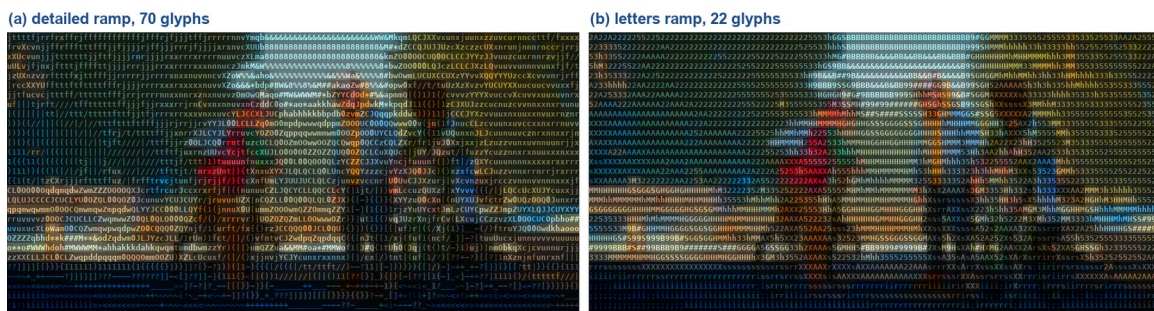


Figure 9. The same frame with the detailed and letters ramps (RGB ASCII, style rich)

4.5 Step 4: The Square-Root Lift and the Three Styles

The problem: brightness counted twice

In colour mode a plain ramp uses brightness **twice**. A dark pixel gets a sparse glyph (little ink) and a dark colour. What the eye perceives is roughly the product of the two, ink coverage times colour intensity, so a pixel at 20 % brightness is displayed at about $0.2 \times 0.2 = 4\%$. Shadows, night scenes and dark clothing disappear into black.

The solution: split brightness between glyph and colour

If the glyph carries $\sqrt{b}$ and the colour carries $\sqrt{b}$, their product is exactly b , the true brightness, and neither factor is crushed. A 20 % pixel now gets a glyph at the 45 % density step and a colour at 45 % intensity.

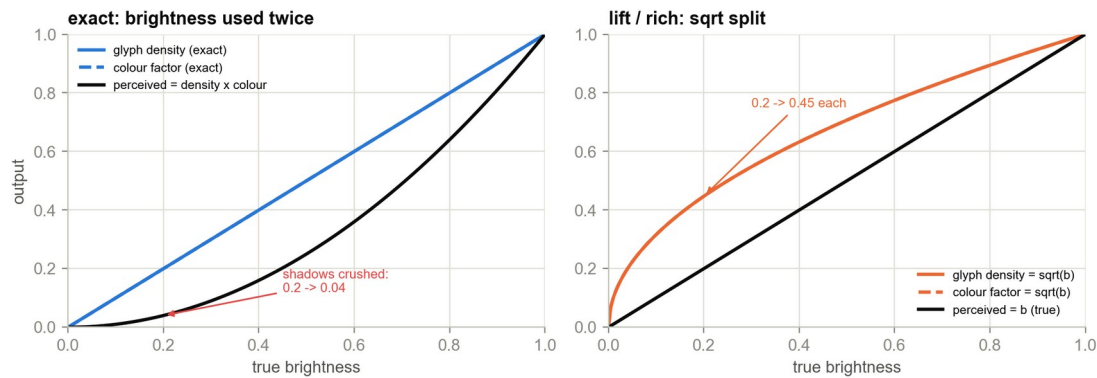


Figure 10. Why the square-root lift works: exact squares the brightness, lift reproduces it

The colour half is implemented by scaling the pixel's colour up by $k = 1/\sqrt{b}$, capped so that no channel exceeds 255 and the hue is preserved:

```
root = sqrt(lum / 255)
k    = min( 1 / root, 255 / max(R, G, B) )
fg   = (R*k, G*k, B*k)
```

Table 12. Styles (the s key)

Style	Glyph from	Foreground	Background	Use
exact	lum/255	pixel colour	none	Plain mapping, true to raw brightness
lift	$\sqrt{(\text{lum}/255)}$	colour $\times$ k	none	Readable dark scenes, black gaps between glyphs
rich (default)	$\sqrt{(\text{lum}/255)}$	colour $\times$ k	colour $\div 2$	Fills the gaps; reads as a picture rather than text

The `rich` style also paints each cell's background with the pixel colour at half strength. That closes the black gaps between characters and between rows (terminals leave line spacing), which is what makes the output look like a continuous image. Figure 11 measures the difference on a dark scene. Under `exact`, 2,946 of 3,600 cells (82 %) are blank or a single dot. Under `lift`, only 675 are (19 %).

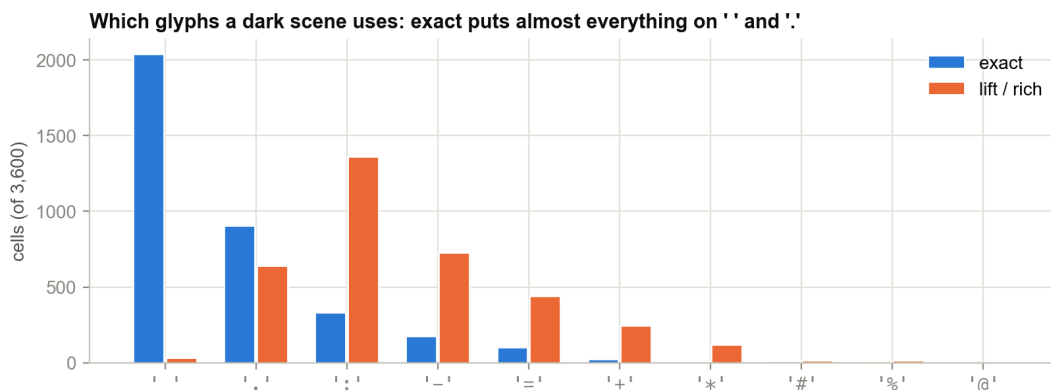


Figure 11. Glyph usage in a dark scene under exact and lift



Figure 12. A dark interior scene in the three styles: exact loses the picture, lift recovers it, rich fills the gaps

4.6 Step 5: Colour Quantisation

Before a colour is emitted, each channel is rounded down to a multiple of $k_{\text{Quant}} = 4$: $c = c / 4 \times 4$. The eye cannot distinguish colours 4 levels apart, but the change makes neighbouring pixels share exactly the same colour far more often. Because the renderer only emits a colour escape when the colour changes (Section 5.6), quantisation directly shrinks the frame. The full per-cell pipeline for RGB ASCII in the rich style is shown below.

Glyph path (blue) and colour path (orange) for one cell in RGB ASCII, style = rich

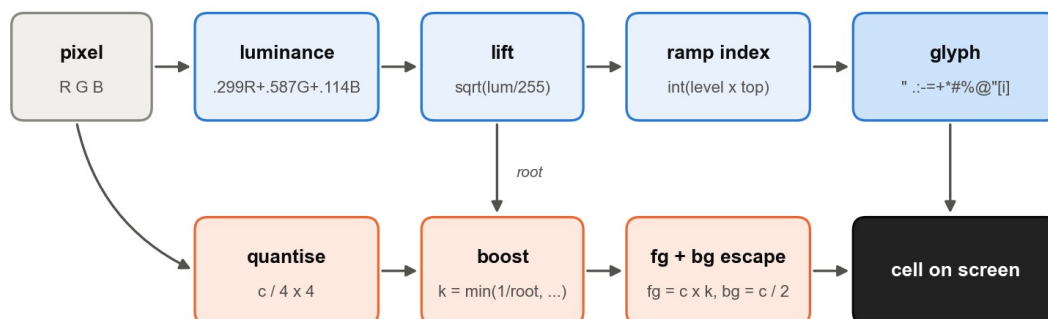


Figure 13. Per-cell pipeline in RGB ASCII (rich): glyph path and colour path

Worked example: the complete cell

Pixel (220, 164, 108): quantised to (220, 164, 108); root = 0.8269; $k = \min(1/0.8269 = 1.209, 255/220 = 1.159) = 1.159$.

fg = (255, 190, 125); bg = (110, 82, 54); glyph #.

Bytes emitted: `ESC[38;2;255;190;125;48;2;110;82;54m#`, 35 bytes including the glyph. The foreground and background are folded into one escape sequence to save 3 bytes per colour change.

Dark pixel (40, 30, 25): lum 32.4. exact gives . in (40, 28, 24), nearly invisible. lift/rich gives - in (112, 78, 67) on (20, 14, 12), clearly visible and still the right hue.

4.7 The Half-Block Technique (RGB BLOCKS)

Unicode character U+2580 ■ (UPPER HALF BLOCK) fills the top half of its cell with the foreground colour and leaves the bottom half to the background colour. Setting both colours gives each character cell **two independent pixels**, and since a cell is about twice as tall as it is wide, each half is roughly square. For cell (x, y) the renderer uses pixel row 2y as the foreground and row 2y+1 as the background:

```
for each row y:
  moveTo(offY + y + 1, offX + 1)
  for each column x:
    top = frame[2y][x]; bot = frame[2y+1][x]
    if top != lastTop: emit ESC[38;2;top m
    if bot != lastBot: emit ESC[48;2;bot m
    emit "\xE2\x96\x80"          // '■' in UTF-8, 3 bytes
```

No letters are involved, so this is not ASCII art. It is the benchmark for image quality, showing the most faithful picture the grid can carry. It is also the most expensive mode in bytes, because nearly every cell changes two colours (Chapter 10).

4.8 Shape Matching (RGB SHAPES)

The problem with a single brightness per cell

A ramp picks a character from the cell's **average** brightness and discards everything happening inside the cell. A cell that is bright on top and dark at the bottom averages to mid-grey and gets the same + as a uniformly grey cell. Every edge in the picture is blurred to the width of a character.

The idea: match the glyph's shape

Characters have shapes. `_` has all its ink at the bottom, `"` at the top, and `/` and `\` run diagonally. If each cell is sampled on a **2-wide × 4-tall sub-grid** (8 samples, roughly square sub-pixels given the cell's 1:2 shape), the renderer can pick the glyph whose **real ink distribution** best matches the 8 samples.

Measuring the glyphs

`glyphs.cpp` holds a table of 95 printable ASCII characters. For each one it stores how much ink the glyph puts in each of the 8 sub-cells, measured by rendering it in Menlo and normalised so that the densest glyph (`N`) has a mean of 1.0. The table is sorted by mean ink, so reading down the list gives a 95-step brightness ramp for free (Figure 15). Figure 14 visualises the measurements for twelve glyphs, and Appendix B lists the full table.

Ink per 2 × 4 sub-cell (darker blue = more ink), from `glyphs.cpp`

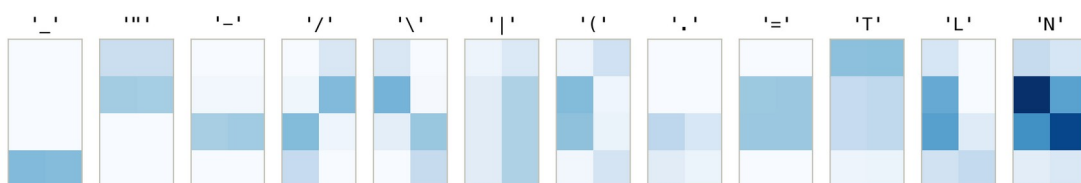


Figure 14. Measured ink per 2 × 4 sub-cell for twelve glyphs

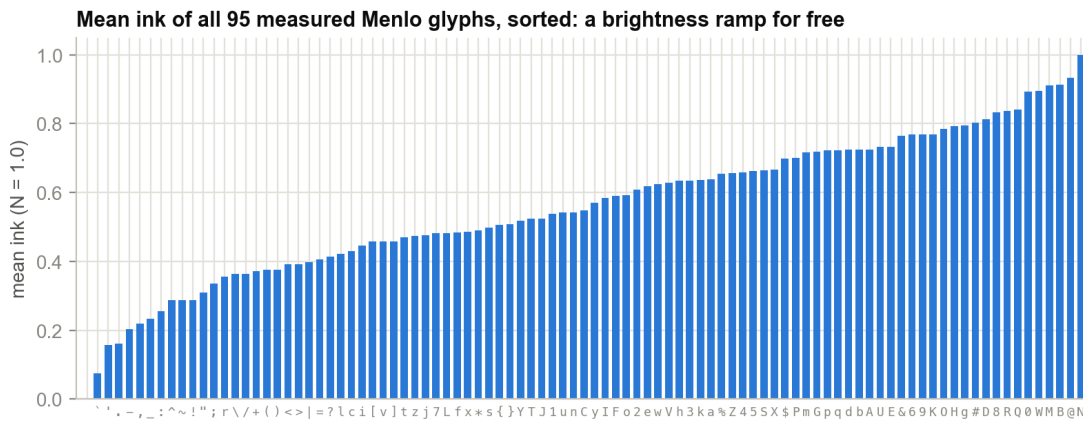


Figure 15. Mean ink of all 95 glyphs, sorted

Why one score is not enough

The obvious approach is to score each glyph by the squared difference between its 8 ink values and the cell's 8 brightness values, but it fails. Nearly every glyph leaves the top and bottom of its cell empty: there is leading above the capitals and descender space below the baseline. A uniformly lit cell therefore matches no glyph well, and every flat region of the picture collapses onto the same two or three characters. The solution is to score two things separately:

- **Level error:** how far the glyph's mean ink is from the cell's mean brightness.
- **Shape error:** how far the glyph's ink *deviations from its own mean* are from the cell's deviations from *its* mean.

```
score(g) = 4 * 8 * (m - gMean[g])2 + w_plane * Σi ( ti - gDev[g][i] )2

m      cell mean brightness, 0 ... 1
ti    +c_plane if sub-pixel i is brighter than the cell mean, else -c_plane
gDev   glyph ink minus the glyph's own mean
planes: 0 (flat)  c = 0.00, w = 0.0    1 (soft)  c = 0.18, w = 0.6    2 (edge)  c = 0.40, w
= 1.0
```

Listing 5. The scoring function used by `buildShapeLut()`.

A flat cell has no deviations, so the level term decides and flat regions get a smooth brightness ramp. A cell with a strong edge has large deviations, so the shape term decides and the glyph traces the edge. The **plane** selects how much weight shape gets, based on the cell's measured contrast:

Table 13. Contrast planes

Plane	Condition (average $ \text{lum} - \text{mean} $)	Shape weight	Behaviour
0 flat	≤ 14	0.0	Pure brightness ramp: smooth gradients
1 soft	15 ... 40	0.6	Shape nudges the choice
2 edge	> 40	1.0	Full shape matching: edges and diagonals

The lookup table

Scoring 95 glyphs $\times$ 8 sub-cells for every cell of every frame is far too slow (Section 10.4 quantifies it). But the inputs that decide the answer are few:

- the plane (3 possibilities);
- the cell mean, quantised to 32 levels;
- the **signature**: which of the 8 sub-pixels are brighter than the mean, an 8-bit number (256 possibilities).

That is only $3 \times 32 \times 256 = 24,576$ distinct questions. `buildShapeLut()` answers all of them once, the first time SHAPES mode is used, and stores the answers in a 24 KB **char** array. After that, classifying a cell is a handful of integer operations and one array read:


```

char glyphFor(const int* lum) { // lum[8], 0..255
    int mean = (lum[0] + ... + lum[7]) / 8;
    int sig = 0, spread = 0;
    for (i = 0; i < 8; ++i) {
        if (lum[i] > mean) sig |= 1 << i; // 8-bit bright/dark pattern
        spread += abs(lum[i] - mean);
    }
    int plane = spread/8 > 40 ? 2 : spread/8 > 14 ? 1 : 0;
    int lvl = mean * 31 / 255;
    return g_shapeLut[plane][lvl][sig];
}

```

Listing 6. Classifying one cell (simplified from `shape_lut.cpp`).

Figure 16 follows one real cell through `glyphFor()`. Figure 17 shows what the table contains: for simple bright/dark patterns it chooses `_` for a bright bottom row, `T` for a bright top, `[` and `L` for bright columns, `(` and `)` for diagonals. The same pattern maps to a denser glyph as the level rises.

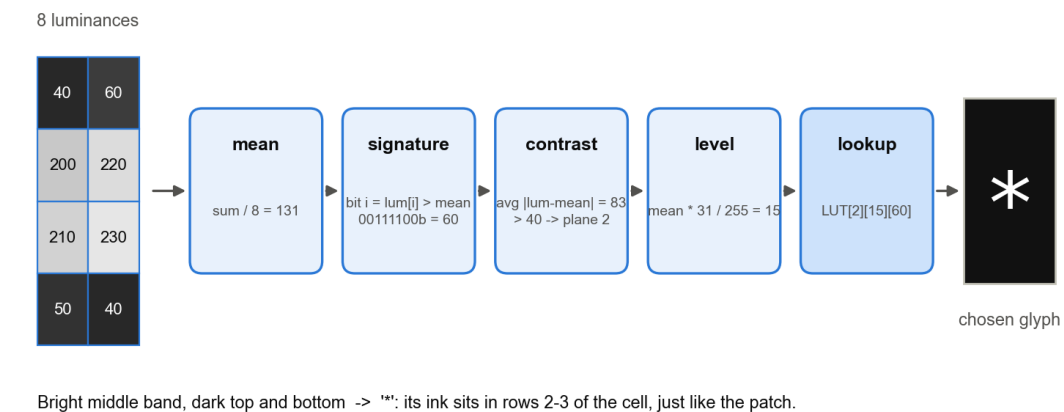


Figure 16. One cell through `glyphFor()`: 8 luminances to the glyph '*'

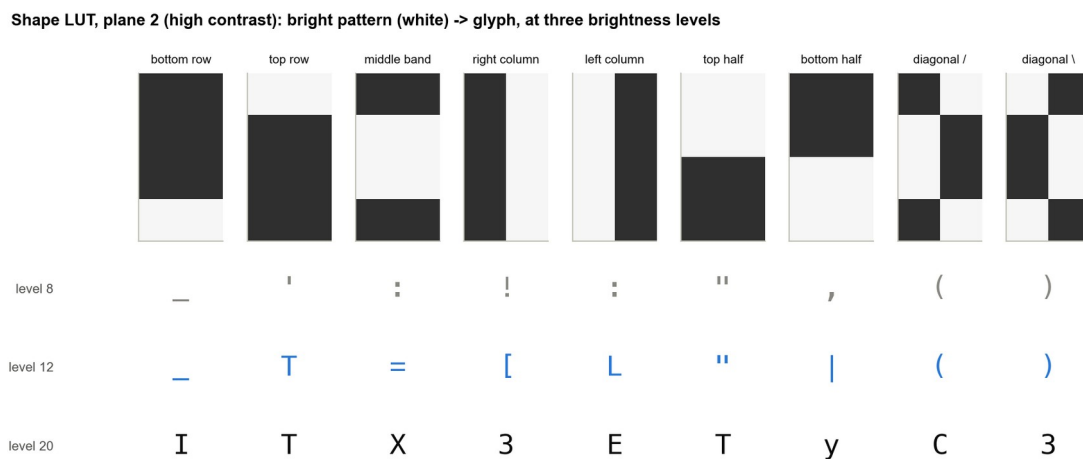


Figure 17. Contents of the lookup table for nine patterns at three brightness levels

On a real frame, most cells are flat (Figure 18), so the SHAPES mode behaves like a fine 95-step ramp over smooth areas and switches to shape matching only where there are edges. Figure 19 shows which glyphs it chose for the bike frame.

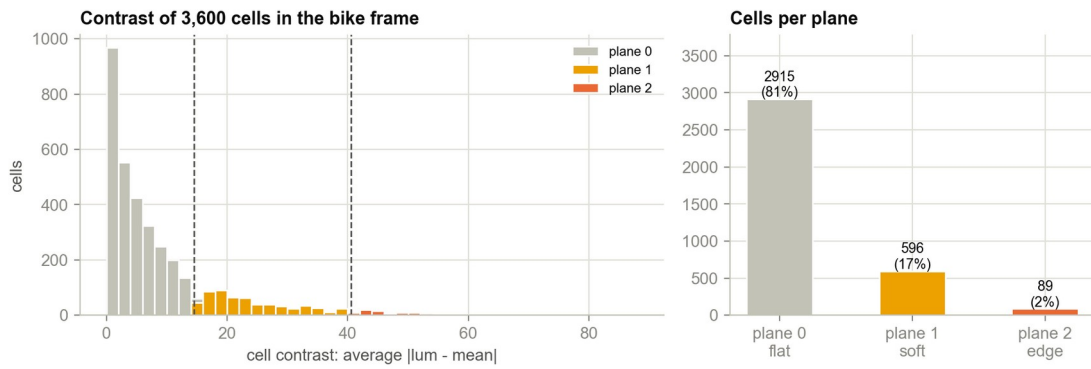


Figure 18. Contrast of every cell in the bike frame, and cells per plane

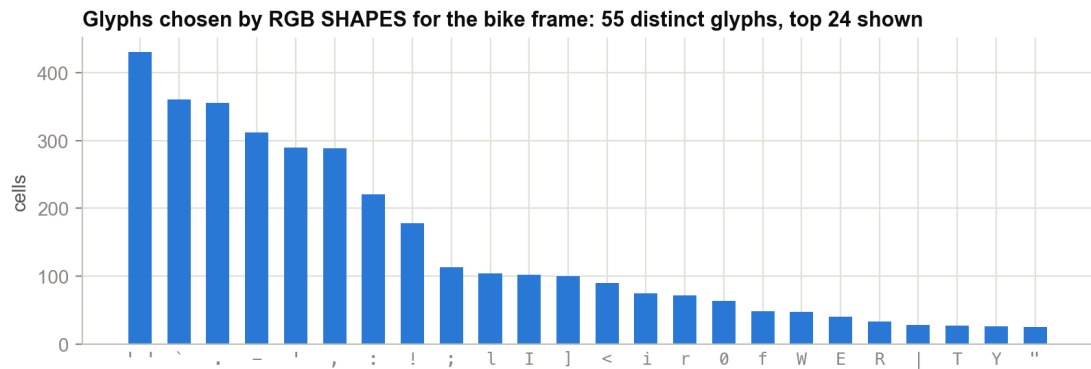


Figure 19. Glyphs chosen by RGB SHAPES for the bike frame

4.9 Visual Comparison on a Real Frame

Figure 20 shows one frame of the test film (a daylight scene with strong colours, fine detail and large flat areas) in every mode at 120 columns, large enough to read individual characters. BLOCKS gives the most faithful image, RGB ASCII rich reads as a textured picture, MONO ASCII carries shape and tone with glyphs alone, and SHAPES follows edges such as the tree line and the bicycle frames with capitals and brackets.



Figure 20. One frame in every mode at 120×30 cells

Figure 21 shows the same frame at 320 columns, the density the player actually uses with a 5 pt font, and Figure 22 enlarges a region so that the individual characters are visible.

(a) RGB ASCII rich, 320 x 80 cells



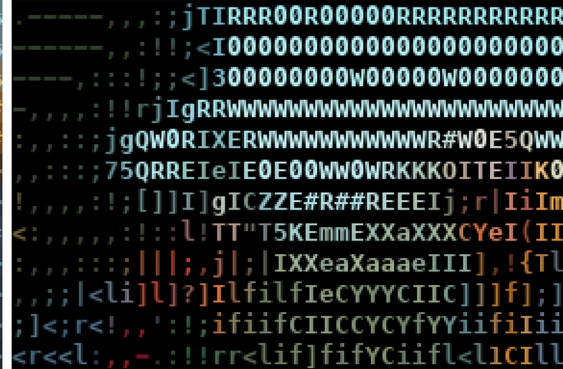
(b) RGB SHAPES, 320 x 80 cells

**Figure 21.** The frame at playback density (320 × 80 cells)

(a) RGB ASCII rich



(b) RGB SHAPES

**Figure 22.** Enlarged detail: ramp glyphs on coloured backgrounds (a) versus shape-matched glyphs (b)

4.10 Algorithm Complexity

Table 14. Cost of each conversion step

Operation	Per frame	Per cell	Notes
Ramp (RGB/MONO ASCII)	O(cells)	1 luminance, 1 sqrt, 1 index	Plus colour boost in lift/rich
Half-block (BLOCKS)	O(cells)	2 colour compares	No glyph decision at all
Shape match, direct	O(cells × 95 × 8)	760 multiply-adds	Not used at run time
Shape match, LUT	O(cells)	8 compares + 1 table read	LUT built once: 3·32·256·95 = 2.33 M glyph scores

Operation	Per frame	Per cell	Notes
Delta test (ramp modes)	O(cells)	1 int compare (+ tolerance test)	Skips unchanged cells entirely

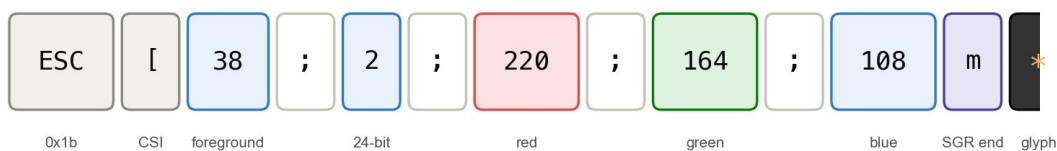
5. Rendering Methods

Chapter 4 decided *what* each cell should show. This chapter covers *how* that reaches the screen. The renderer turns one frame into one `std::string` of text and escape codes and writes it in a single call. Almost all the engineering here is about making that string as short as possible without changing what the viewer sees.

5.1 The Terminal as a Display Device

A terminal emulator parses the bytes a program writes. Printable characters are drawn at the cursor position and advance it; escape sequences change state such as cursor position and colours. The player uses the sequences listed in Appendix C. The one written most is the 24-bit colour **SGR** (Select Graphic Rendition) sequence:

One coloured character on screen = 17 to 20 bytes:



The same bytes written by `setColor(out, false, 220, 164, 108)` followed by `out += '*'`

Figure 23. Anatomy of one coloured character: a true-colour SGR sequence followed by the glyph

A coloured character typically costs 16–19 bytes of escape code plus 1 byte of glyph. With 25,600 cells per frame and 20 frames per second, how often that escape is emitted decides whether the terminal can keep up.

5.2 Building a Frame as One String

`ansi.h` provides three header-only helpers. They are called once per cell, hundreds of thousands of times a second, so they are declared `inline` in the header where the compiler can see through them:

```
inline void appendInt(std::string& s, int v) {           // no printf, no allocation
    char b[12];
    auto r = std::to_chars(b, b + sizeof b, v);
    s.append(b, r.ptr);
}
inline void moveTo(std::string& s, int row, int col) { // ESC[row;colH (1-based)
    s += "\x1b["; appendInt(s, row); s += ';'; appendInt(s, col); s += 'H';
}
inline void setColor(std::string& s, bool bg, int r, int g, int b) {
    s += bg ? "\x1b[48;2;" : "\x1b[38;2;";
    appendInt(s, r); s += ';'; appendInt(s, g); s += ';'; appendInt(s, b); s += 'm';
}
```

Listing 7. The escape-code helpers in `ansi.h`.

The frame string `out` is a member of `Player` and is cleared, not destroyed, between frames, so its capacity is reused and rendering allocates nothing. `writeAll()` then loops on `write()` until every byte has been accepted, because a write to a terminal can accept fewer bytes than requested, and dropping the remainder would tear the picture.

5.3 Synchronized Updates

If the terminal repaints while a frame is only half written, the viewer sees the top of the new frame over the bottom of the old one (tearing). Every frame is therefore wrapped in `ESC[?2026h ... ESC[?2026l`, the synchronized-update mode. Terminals that support it buffer everything in between and show it in one step; terminals that do not simply ignore the sequence.

5.4 The Three Render Loops

`Player::render()` has one branch per family of modes:

- **Blocks:** row by row, emit a foreground escape only when the top pixel's colour differs from the previous cell's, a background escape only when the bottom pixel's differs, then the 3-byte `ESC[?2026h`. Every cell is redrawn every frame.
- **Shapes:** row by row, gather each cell's 8 sub-pixel luminances and colour sum, call `glyphFor()`, and emit the average colour only when it changes. Spaces carry no colour. Every cell is redrawn every frame.
- **Ramp (RGB and MONO ASCII):** the most optimised path, described in the rest of this chapter. It redraws only cells that changed and keeps the frame within a byte budget.

5.5 Delta Rendering: Redraw Only What Changed

Consecutive video frames are mostly similar. The ramp renderer keeps `drawn`, one `int` per cell recording what that cell currently shows on screen, packed as `colour << 8 | glyph` (with `-1` meaning unknown). For each cell of a new frame it computes the glyph and quantised colour, then:

```
old = drawn[cell]
if old >= 0 and glyph(old) == ch and (colour(old) == c or near(c, colour(old), tol)):
    continue // already on screen: emit nothing, not even a cursor move
if cursor != x: moveTo(row, x) // jump only when the previous cell was skipped
... emit colour if needed, then the glyph ...
drawn[cell] = c << 8 | ch; cursor = x + 1
```

Listing 8. The per-cell delta test in the ramp renderer.

Two savings follow. Unchanged cells cost nothing, and a run of changed cells needs only one cursor move at its start, because printing a character advances the cursor by itself. `drawn` is reset to `-1` whenever the screen is cleared (mode change, style change, resize, seek), which forces a full redraw.

5.6 Colour Reuse

The last colour emitted stays active in the terminal until it is changed. The renderer remembers it (`last`) and emits a new SGR sequence only when the next inked cell's colour differs. Together with the quantisation to steps of 4 (Section 4.6), this skips a large share of colour escapes in flat regions such as sky or walls.

5.7 The Byte Budget and Adaptive Colour Tolerance

Why a budget is needed

Terminal.app accepts roughly 5 MB/s. At 20 fps that is about **250 KB per frame**; more than that and `write()` blocks, the terminal falls behind, and the viewer sees a stall. Delta rendering alone is not enough. Video compression noise changes most pixels slightly from frame to frame, so with exact colour comparison almost every cell counts as changed. Section 10.2 shows that exact deltas save only 17 %.

Colour tolerance

When a frame is too big, colours within `tol` of what is already on screen (per channel) are treated as equal, both for the delta test and for colour reuse. `tol` starts at 0 (exact) and changes only under pressure:

- If a frame came out at more than $1.25 \times \text{budget}$, it is **discarded and rebuilt immediately** with `tol = max(2 * tol, tol + 8)`, up to 3 retries. A scene cut changes every cell at once; letting `tol` creep up over the following second would make the stall visible, while an immediate retry hides it.

- After each frame: over budget $\rightarrow \text{tol} += 2$; under half the budget $\rightarrow \text{tol} -= 1$. Tolerance rises quickly and falls back slowly, so the picture returns to exact colour when the scene calms down.
- `tol` never exceeds `kMaxTol = 64`.

Adapting the budget itself

The budget starts at 180 KB and moves between 40 KB and 400 KB according to how long `writeAll()` took. If the write blocked for more than half a frame period (25 ms at 20 fps), the terminal is falling behind and the budget drops by 30 %. If the write took under a fifth of a frame and the frame was not tiny, the budget grows by 1 %. The asymmetry is deliberate: a stall is obvious to the viewer, a slightly lower colour precision is not.

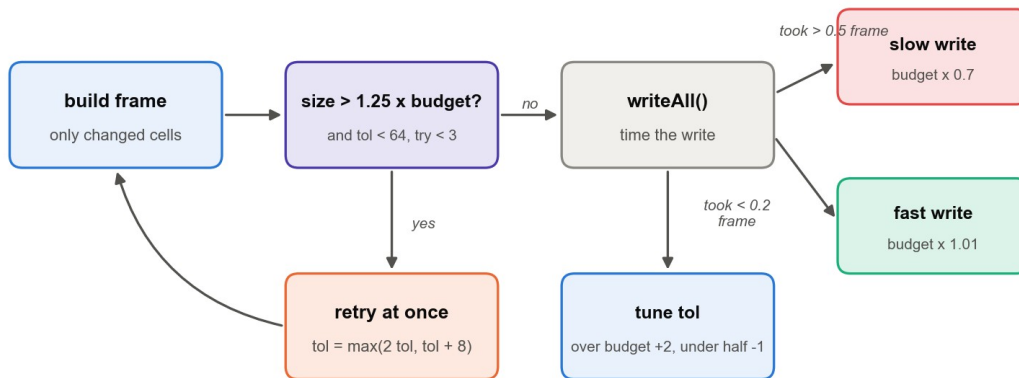


Figure 24. The byte-budget feedback loop in `Player::render()`

Figure 25 shows the effect on six seconds of the test film at 320×80 cells. The full redraw is 500–700 KB every frame. Exact deltas help only in still moments, such as the slow pan near 4.8 s. The player's combined method stays close to the 180 KB budget throughout and averages 125 KB. The colour tolerance the renderer chose along the way rose to at most 30 during busy passages and fell back towards exact colour when the scene allowed.

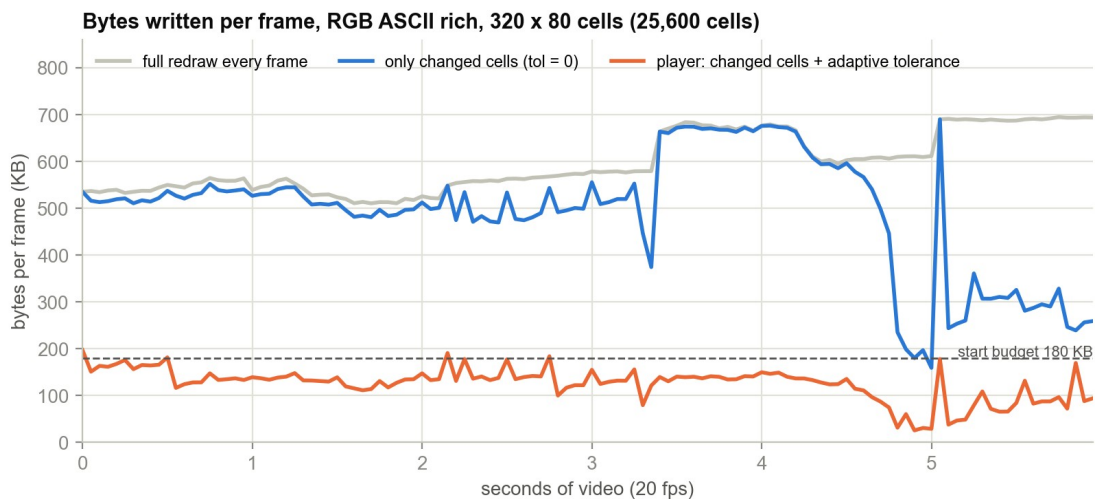


Figure 25. Bytes per frame over six seconds of video for three rendering strategies (measured)

5.8 Status Bar

The last terminal row shows a status bar drawn into the same frame string. It contains a play/pause marker, the current time and duration, a progress bar drawn with `—` and `-` in two colours, the active mode, charset and style, the audio state, and key hints. Because terminals vary in width, `drawStatus()` holds nine hint strings from longest to shortest and uses the longest one that fits whole alongside at least 8 cells of progress bar. A hint cut off mid-word is worse than a shorter one.


```
> 07:12 / 30:54 [RGB ASCII classic rich] sound | spc
pause <-/-> 5s up/dn 30s m mode c chars s style -/+ text size a audio q back
```

Listing 9. The status bar on a wide terminal (colours omitted).

6. Module Descriptions

The program is divided into 15 modules. Each is described below using the same template: its purpose, its job in the player, its interface, how it works internally, and a concrete example. Table 15 summarises them and Figure 26 compares their sizes.

Table 15. Module summary

Module	Area	One-line job
common	core	Steady clock helpers and time formatting
terminal	terminal	Raw mode, alternate screen, signals, window size, reliable writes
input	terminal	Decode keypresses, including multi-byte arrow keys, with a timeout
ansi	terminal	Inline builders for cursor and colour escape codes
shell	platform	Safe shell quoting, running commands, finding tools
zoom	platform	macOS Terminal.app font zoom and cell-aspect measurement
probe	media	Read video size, frame rate, duration, audio track via ffmpeg
audio	media	Start/stop the ffmpeg -> ffmpeg audio process group
modes	render	Mode/Style enums, ramps, and all rendering tuning constants
glyphs	render	Measured ink table for 95 ASCII glyphs
shape_lut	render	Build the 24 KB lookup table; classify a 2 × 4 cell
player	player	Decode, synchronise, render: the main loop
browser	ui	Full-screen file picker
message	ui	Full-screen message / error page
main	entry	Start-up checks, Ghostty launcher, browse → play loop

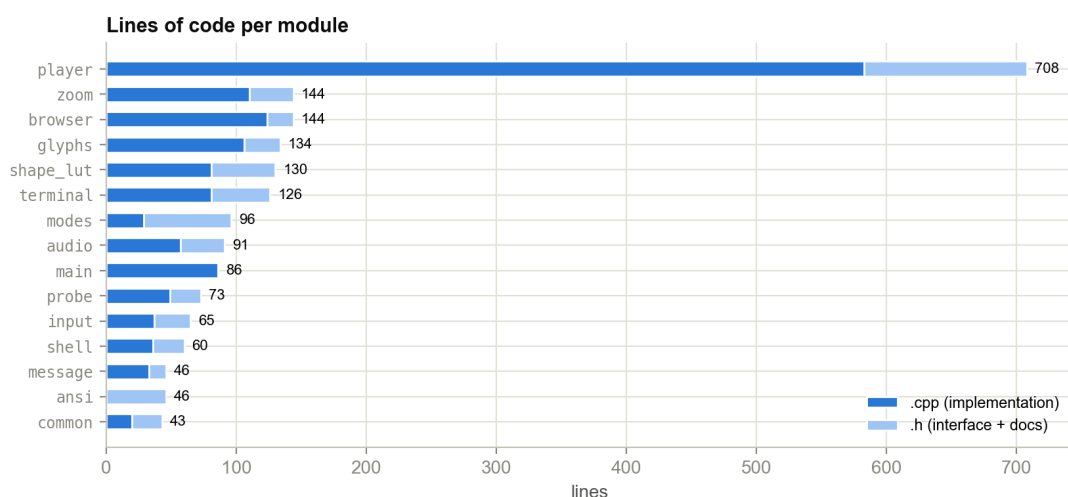


Figure 26. Lines of code per module (implementation and interface)

6.1 core/common: The Clock

Files: include/asciplayer/common.h, src/core/common.cpp

Purpose: Define the single clock the whole player is timed against.

Job in the player: Every "is the next frame due?" and "has the sound started?" question uses `secondsSince()`; the status bar uses `fmtTime()`.

Function / type	What it does
Clock	Alias for <code>std::chrono::steady_clock</code>
<code>secondsSince(t)</code>	Seconds elapsed since time point <code>t</code> , as a double
<code>fmtTime(t)</code>	Format seconds as "MM:SS" or "H:MM:SS"

How it works

A steady clock only ever moves forward at one second per second. The wall clock (`system_clock`) can jump when the user or NTP changes the time, which would make the film leap. `secondsSince()` subtracts two time points and converts the duration to double seconds. `fmtTime()` shows hours only when there are any, because `01:23` reads better than `00:01:23` for a short clip.

Example

`fmtTime(83.4)` returns "`01:23`"; `fmtTime(3725)` returns "`1:02:05`". In the main loop, `secondsSince(t0) < framesRead / fps` is the test for "the next frame is not due yet".

6.2 terminal/terminal: Taking Over the Terminal

Files: `include/ascioplayer/terminal.h`, `src/terminal/terminal.cpp`

Purpose: Put the terminal into "game mode" for the duration of the program and always put it back.

Job in the player: Provides raw mode, the alternate screen, reliable output, the window size, and the signal flags the main loop polls.

Function / type	What it does
<code>enableRaw()</code>	Save <code>termios</code> ; clear <code>ECHO</code> and <code>ICANON</code> ; <code>VMIN = VTIME = 0</code> ; alternate screen; hide cursor; no wrap
<code>restoreTerminal()</code>	Undo zoom, restore <code>termios</code> , reset colours, show cursor, leave alternate screen; safe to call twice
<code>writeAll(s)</code>	Loop on <code>write()</code> until every byte is written; retry on <code>EINTR</code>
<code>termSize(cols, rows)</code>	<code>ioctl(TIOCGWINSZ)</code> ; falls back to <code>80 × 24</code>
<code>installSignalHandlers()</code>	<code>SIGINT/SIGTERM</code> → <code>g_quit</code> ; <code>SIGWINCH</code> → <code>g_resized</code> ; ignore <code>SIGPIPE</code>
<code>g_quit</code> , <code>g_resized</code>	volatile <code>sig_atomic_t</code> flags set by the handler

How it works

By default a terminal is in **canonical mode**. It collects typed characters into a line, echoes them, allows editing, and hands the line to the program only on Enter. A player needs each key immediately and nothing echoed. `enableRaw()` clears `ICANON` and `ECHO` and sets `VMIN = VTIME = 0`, so `read()` returns at once even when no key was pressed. It also switches to the **alternate screen** (`ESC[?1049h`), so the user's scrollbar is untouched and comes back on exit, as with vim.

A signal handler may safely do almost nothing (no allocation, no I/O, no locks), so `onSignal()` only sets a flag, and the main loop does the real work on its next pass. `SIGPIPE` is ignored so that writing to a closed ffmpeg pipe returns an error instead of killing the player.

Example

```
// what enableRaw() writes after changing termios:
ESC[?1049h  enter alternate screen
ESC[?25l    hide cursor
ESC[?7l     disable auto-wrap (a glyph in the last column must not scroll)
ESC[2J      clear
```

6.3 terminal/input: Reading Keys

Files: include/asciplayer/input.h, src/terminal/input.cpp

Purpose: Turn raw keyboard bytes into a single Key value.

Job in the player: Used by the player, the browser and the message page. Its timeout makes it the player's frame timer.

Function / type	What it does
enum class K	None, Up, Down, Left, Right, Enter, Back, Esc, PgUp, PgDn, Char
struct Key { K k; char c; }	The key; c is set only for K::Char
readKey(timeoutMs)	>0 wait that long; 0 poll; <0 block until a key or a signal

How it works

Arrow keys send three bytes: Up is `ESC [ A`. `readKey()` uses `poll()` to wait for the first byte. If it is ESC, it reads up to two more bytes, each with a **25 ms** timeout. The timeout distinguishes the Escape key (one byte, nothing follows) from an arrow key (all three bytes arrive together). Without it the program would block forever after a real Escape. PageUp and PageDown are four bytes (`ESC [ 5 ~`), so the trailing `~` is also consumed. Enter arrives as `\r` or `\n`, Backspace as 127 or 8.

Example

Bytes received	Key returned
1B 5B 41	K::Up
1B 5B 43	K::Right
1B 5B 35 7E	K::PgUp
1B (nothing within 25 ms)	K::Esc
0D	K::Enter
20	K::Char, c = ' ' (pause)

6.4 ansi.h: Escape-Code Builders

Files: include/asciplayer/ansi.h (header only)

Purpose: Append cursor-movement and colour escape codes to a string as cheaply as possible.

Job in the player: Every character the player or browser draws is positioned and coloured through these three functions.

Function / type	What it does
appendInt(s, v)	Append decimal digits of v using <code>std::to_chars</code>
moveTo(s, row, col)	Append <code>ESC[row;colH</code> (1-based)
setColor(s, bg, r, g, b)	Append <code>ESC[38;2;r;g;b</code> m or <code>ESC[48;2;r;g;b</code> m

How it works

The functions append to a caller-owned string instead of returning new strings, so no temporaries are created. `std::to_chars` writes digits straight into a stack buffer with no locale or format-string parsing, unlike `printf` or `std::to_string`. Keeping the file header-only lets the compiler inline all three into the render loop.

Example

`moveTo(out, 5, 12); setColor(out, false, 255, 190, 125); out += '#';` appends the 27 bytes `ESC[5;12H ESC[38;2;255;190;125m #`.

6.5 platform/shell: Running Other Programs

Files: include/ascioplayer/shell.h, src/platform/shell.cpp

Purpose: Run external commands safely and capture their output.

Job in the player: Used to run `ffprobe`, `ffmpeg`, `osascript` and to check that tools are installed.

Function / type	What it does
<code>shq(s)</code>	Wrap <code>s</code> in single quotes, escaping embedded quotes as <code>'\"'</code>
<code>runCmd(cmd)</code>	<code>popen(cmd)</code> , read all stdout, <code>pclose</code> ; retry reads interrupted by signals
<code>haveTool(name)</code>	<code>command -v name</code> ; true if on the PATH

How it works

File names reach a shell command line, and they can contain spaces, `$`, `;`, backquotes and quotes. Pasting one unquoted would let a crafted file name run arbitrary commands. `shq()` wraps every user-supplied string in single quotes, inside which the shell interprets nothing, and replaces each embedded `'` with `'\''` (close quote, escaped quote, reopen). It is the project's one security-critical function.

`runCmd()` retries a read interrupted by `EINTR`. That is necessary because the font zoom resizes the window, which sends `SIGWINCH` to the player while `osascript`'s output is being read.

Example

```
shq("Bob's clip; rm -rf ~/.mp4")
-> 'Bob'\''s clip; rm -rf ~/.mp4'           (one harmless literal argument)
```

6.6 platform/zoom: Smaller Characters on macOS

Files: include/ascioplayer/zoom.h, src/platform/zoom.cpp

Purpose: Increase picture resolution by shrinking the terminal font during playback, and report the true cell shape.

Job in the player: `Player::run()` calls `zoomEnter()` before the first frame and `zoomLeave()` after the last; `-` and `+` call `zoomStep()`; `layout()` calls `cellAspect()`.

Function / type	What it does
<code>zoomEnter()</code>	Remember the tab's font and size; switch to Menlo-Bold at <code>ASCIIPLAYER_ZOOM</code> pt (default 5)
<code>zoomLeave()</code>	Restore the original font and size
<code>zoomStep(±1)</code>	Change the size by one point, between 2 pt and the original
<code>zoomActive()</code>	True while zoomed (the status bar shows the <code>-/+</code> hint)
<code>cellAspect()</code>	Cell width ÷ height from pixel size, measurement, or 0.5

How it works

Everything here is specific to Terminal.app and is skipped unless `TERM_PROGRAM` is `Apple_Terminal`, so the rest of the player can call it unconditionally. An AppleScript finds the tab whose `tty` matches the player's own terminal device, reads its font, and sets a new font name and size. Terminal sometimes shrinks the window instead of adding cells, so the script re-applies the window bounds to force a re-grid. It then returns the window size and the new row and column counts. From these the true cell aspect is computed (subtracting a 28-pixel title bar). Menlo-Bold is used because heavier strokes put more ink in each cell and its line height is tighter.

Changing the font resizes the grid, which raises `SIGWINCH`. The main loop then re-lays out and restarts the decoder at the new size automatically.

Example

Going from a 12 pt profile font to the default 5 pt multiplies the cell count in each direction by about $12/5 = 2.4$, so the same window holds roughly 5.8 times as many picture samples. `ASCIIPLAYER_ZOOM=0` `./asciiplayer` disables the zoom entirely.

6.7 media/probe: What Is in the File

Files: `include/asciiplayer/probe.h`, `src/media/probe.cpp`

Purpose: Find out the video's size, frame rate, duration and which audio stream to play.

Job in the player: Called once at the start of `Player::run()`; the result drives layout, timing, the progress bar and whether sound is attempted.

Function / type	What it does
<code>struct VideoInfo</code>	w, h, fps (default 25), duration (0 = unknown), audio, audioIndex
<code>probe(path)</code>	Run <code>ffprobe</code> twice and parse the output

How it works

The first `ffprobe` call asks only for `width`, `height`, `r_frame_rate` and `duration` of the first video stream, printed as `key=value` lines, which keeps parsing to a simple split on `=`. The frame rate arrives as a fraction such as `24000/1001` and is divided out. Values outside 1–120 fps are treated as unreliable and replaced with 25. The second call lists every audio stream as `index,default` pairs. The stream flagged `default` wins; otherwise the first is used. For a dual-language release with Hindi as the default track this chooses Hindi, as a regular player would.

Example

```
ffprobe ... -select_streams a -show_entries stream=index:stream_disposition=default -of csv=p=0
1,1      <- stream 1 is the default audio track -> audioIndex = 1
2,0
```

6.8 media/audio: The Sound Pipeline

Files: `include/asciiplayer/audio.h`, `src/media/audio.cpp`

Purpose: Play the soundtrack from an exact timestamp and expose its playback clock.

Job in the player: `Player::startSound()` / `stopSound()` call these on open, seek, pause, resume and mute.

Function / type	What it does
<code>startAudio(path, stream, t, errFd)</code>	fork a process group running <code>sh -c 'ffmpeg ... ffplay ...'</code> ; <code>errFd</code> receives <code>ffplay</code> 's <code>stderr</code>
<code>stopAudio(pid, errFd)</code>	<code>kill(-pid, SIGTERM)</code> , <code>waitpid</code> , close <code>errFd</code>

How it works

The child calls `setpgid(0, 0)` to lead a new process group, sends `stdin` and `stdout` to `/dev/null` and `stderr` to a pipe, closes every other inherited descriptor, and `exec`s `/bin/sh` with the pipeline command. The parent makes the pipe's read end non-blocking and close-on-exec and returns it as the clock feed. The command begins with a **watchdog**, (`while kill -0 $$PID; do sleep 0.5; done; kill -TERM 0`) `&`, which kills the whole group if the player itself dies without cleaning up. Chapter 8 explains why each of these steps is required.

Example

Seeking to 7:12 calls `stopAudio()` (the old group receives `SIGTERM`), then `startAudio(path, 1, 432.0, afd)`. About 0.1–0.4 s later `ffplay` starts printing lines such as `0.42 M-A: 0.000 ...` into `afd`.

6.9 render/modes: Modes, Ramps and Tuning

Files: include/ascioplayer/modes.h, src/render/modes.cpp

Purpose: Define the rendering vocabulary and keep every tuning constant in one place.

Job in the player: The player reads the current Mode and Style, indexes kRamps, and uses the constants for budgets and timing.

Function / type	What it does
enum class Mode	Color, Mono, Shapes, Blocks
enum class Style	Rich, Lift, Exact
kRamps[], kRampCount	The three character ramps (count computed with sizeof)
kQuant = 4	Colour quantisation step
kFrameBudgetStart/Min/Max	180,000 / 40,000 / 400,000 bytes
kMaxTol = 64	Largest colour tolerance
kRampFps = 20, kRampMaxSkip = 5	Ramp-mode frame rate and consecutive-skip limit

How it works

The header documents why each constant has its value. For example, Terminal.app on an M2 kept 20 fps up to about 300 KB per frame, which is why the budget starts at 180 KB with headroom. `kRampCount` is computed from `sizeof`, so adding a ramp needs no other change.

Example

Adding `{ "blocks", " .-:=+*#%@" }` to `kRamps` makes it appear as a fourth charset under the `c` key with no other edits.

6.10 render/glyphs: The Ink Table

Files: include/ascioplayer/glyphs.h, src/render/glyphs.cpp

Purpose: Describe what each ASCII character looks like, numerically.

Job in the player: The input from which the shape lookup table is computed.

Function / type	What it does
struct Glyph { char ch; float ink[8]; }	ink[] in row-major 2×4 order, top row first
kGlyphs[], kGlyphCount	95 entries, sorted by mean ink

How it works

Pure data, measured once by rendering each glyph in Menlo, sampling its coverage on the 2×4 grid, and normalised so that `N`, the densest glyph, has mean 1.0. Values above 1 occur where a glyph is denser than its average in one sub-cell.

Example

```
{ ' ' , { 0, 0, 0, 0, 0, 0, 0.9425, 0.9303 } } // all ink in the bottom row
{ 'N' , { 0.4824, 0.4799, 0.7586, 0.7546, 0, 0, 0, 0 } } // all ink in the top half
```

6.11 render/shape_lut: The Shape Matcher

Files: include/ascioplayer/shape_lut.h, src/render/shape_lut.cpp

Purpose: Pick the glyph that best matches a 2×4 patch of picture, in constant time.

Job in the player: `render()` calls `buildShapeLut()` on entering SHAPES mode and `glyphFor()` for every cell.

Function / type	What it does
<code>kSubX = 2, kSubY = 4, kSubN = 8</code>	Sub-pixel grid per cell
<code>buildShapeLut()</code>	Fill <code>g_shapeLut[3][32][256]</code> ; returns immediately after the first call
<code>glyphFor(lum)</code>	Mean, signature, contrast plane, level → table lookup

How it works

Described in full in Section 4.8. The table is built lazily, so users who never enter SHAPES mode never pay for it. Construction scores 95 glyphs for each of 24,576 entries (2.3 million scores) and takes a few milliseconds.

Example

For the cell `[40, 60, 200, 220, 210, 230, 50, 40]`: mean 131, signature 60, contrast 83 → plane 2, level 15, so the result is `g_shapeLut[2][15][60] = '*'` (Figure 16).

6.12 player/player: The Main Loop

Files: `include/ascioplayer/player.h`, `src/player/player.cpp` (583 lines)

Purpose: Play one video from its first frame to its last.

Job in the player: The top of the program: it owns both decoders, all frame buffers, the timing state and every user action.

Function / type	What it does
<code>run()</code>	probe → zoom → <code>openAt(0)</code> → main loop → cleanup
<code>layout()</code>	Fit the video to the grid (Section 4.2)
<code>openAt(t)</code>	(Re)start the video decoder and sound at time <code>t</code>
<code>readFrame()</code>	fread exactly <code>pw-ph-3</code> bytes; false at end of stream
<code>syncToAudio()</code>	Parse <code>ffmpeg</code> 's clock and move <code>t0</code> (Section 7.4)
<code>render()</code>	Frame → ANSI string → <code>writeAll</code> (Chapter 5)
<code>handleKey(k)</code>	Dispatch a key; false means leave
<code>seek, togglePause, drawStatus, decodeError</code>	User actions, status bar, diagnostics

How it works

`openAt()` is the one place playback (re)starts. Start, seek, resume, resize and mode change all call it, which keeps every restart consistent: close the decoder, stop the sound, clamp `t` to the file, re-run `layout()`, clear the screen if the geometry or mode changed, spawn `ffmpeg` at `t` with the new size, reset the frame counter and `t0`, and restart the sound.

The playback position is derived from the frame count, `startPos + framesRead / fps`, rather than from a clock, so it stays exact even when drawing falls behind. If the decoder ends early, `ffmpeg`'s `stderr` log is read back, repeated lines are collapsed, and the last six are shown. This distinguishes an unsupported or truncated file from a normal end of film.

Example

Pressing `m` during playback: `handleKey` sets the next mode and calls `openAt(position())`. The decoder restarts at the same media time with the new pixel size (for example `644 × 324` for SHAPES instead of `320 × 80`), the screen is cleared, and the next frame is drawn in the new mode. The sound restarts from the same point.

6.13 ui/browser: The File Picker

Files: include/ascioplayer/browser.h, src/ui/browser.cpp

Purpose: Let the user choose a video without typing a path.

Job in the player: Shown at start-up (unless a file is given) and after every video.

Function / type	What it does
browse(dir)	Show the list; return the chosen path or "" on quit; dir is updated in place

How it works

`listDir()` lists the directory with `std::filesystem`, skips hidden entries and unreadable ones, and keeps directories plus files whose extension is one of 14 video types (mp4, mkv, avi, mov, webm, flv, wmv, m4v, mpg, mpeg, gif, ts, 3gp, ogv). Directories come first, then files, each sorted by name, with `..` at the top. The selection scrolls within the visible rows. When moving up to the parent, the directory just left is pre-selected. `readKey(-1)` blocks until a key arrives, and a resize interrupts it so the list is simply redrawn.

Example

Key	Action
Up / Down, PgUp / PgDn	Move selection
Enter or Right	Open directory / play file
Left or Backspace	Parent directory
~	Home directory
q or Esc	Quit

6.14 ui/message: Messages and Errors

Files: include/ascioplayer/message.h, src/ui/message.cpp

Purpose: Show a multi-line message full-screen and wait for a key.

Job in the player: Used when a file cannot be read or decoding stops early.

Function / type	What it does
showMessage(msg)	Clear, print lines from row 2 (clipped to width), "(press any key)", wait

How it works

Lines are clipped rather than wrapped so the layout stays predictable. The wait polls `readKey(100)` so that Ctrl-C (`g_quit`) still exits.

Example

```
Could not decode this video:
~/Downloads/broken.mp4

ffmpeg said:
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x...] moov atom not found
~/Downloads/broken.mp4: Invalid data found when processing input

(press any key)
```

6.15 main.cpp: Start-up

Files: src/main.cpp

Purpose: Check the environment, handle arguments, and run the browse → play loop.

Job in the player: The program's entry point.

Function / type	What it does
main(argc, argv)	isatty checks; ffmpeg/ffprobe check; signals; -g launcher; raw mode; loop

How it works

It refuses to run unless stdin and stdout are terminals, and prints install instructions for Ubuntu, Arch and macOS if ffmpeg or ffprobe is missing. With `-g` (or `--ghostty`) it runs `open -na Ghostty --args --font-size=5 --maximize=true ... -e <self> <file>` and exits, so the player continues in a new, GPU-rendered Ghostty window. Otherwise it enters raw mode, registers `restoreTerminal` with `atexit`, and loops: browse for a file (or use the one given), construct a `Player` in its own scope so its destructor cleans up, run it, and return to the browser.

Example

```
./ascioplayer                # open the file browser in the current directory
./ascioplayer ~/Movies/clip.mp4 # play immediately, browser opens in ~/Movies afterwards
./ascioplayer -g ~/Movies/clip.mp4 # relaunch in a maximised 5 pt Ghostty window
```

7. Playback Engine: Timing, Pacing and Synchronisation

7.1 The Main Loop

`Player::run()` contains the loop that drives playback (Figure 27). Each pass handles a resize if one was signalled, reads the audio clock, waits for either a key or the next frame's due time, and then reads, possibly skips, and renders one frame.

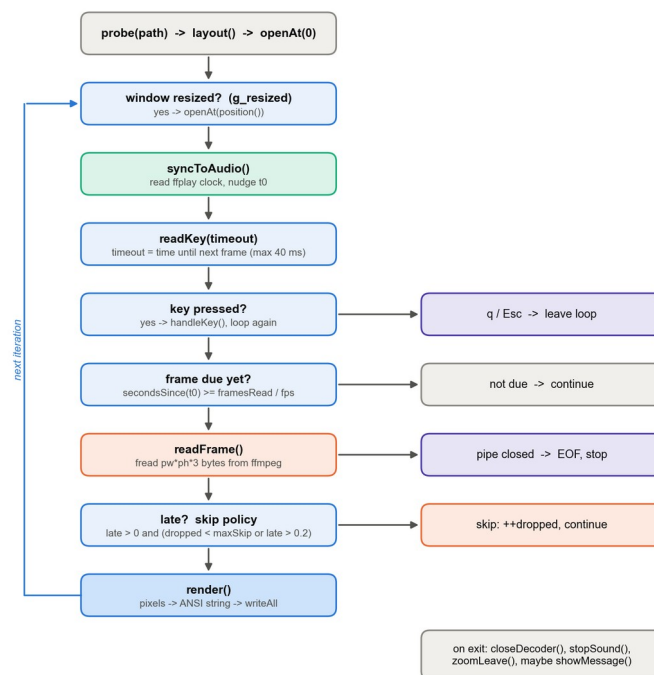


Figure 27. The main playback loop in `Player::run()`

7.2 One Wait for Both Keys and Frames

The player has no timer thread. The main loop calls `readKey(timeout)`, and the timeout is set to the time remaining until the next frame is due, capped at 40 ms so the audio clock is also read regularly:

```
double wait = framesRead / fps - secondsSince(t0);           // seconds until the next frame
int timeout = clamp(int(wait * 1000), 0, 40);                // ms
if (holding) timeout = 10;                                   // waiting for sound: poll faster
Key k = readKey(timeout);
```

A single blocking `poll()` therefore makes keys respond instantly and wakes the loop on time for the next frame, with no threads, locks or busy-waiting. Frame n of the current decoder is due at $t_0 + n / \text{fps}$, where t_0 is the moment the decoder was opened.

7.3 Position from the Frame Count

The current playback position is $\text{startPos} + \text{framesRead} / \text{fps}$, the media time at which the decoder was opened plus the number of frames consumed since. Because it counts frames rather than reading a clock, it is exact regardless of how late drawing runs, and seeking by ± 5 s lands exactly 5 s of media away.

7.4 Audio/Video Synchronisation

Sound is the master clock

A dropped or repeated video frame is almost invisible; a gap or click in the sound is obvious at once. The player therefore never adjusts the audio. It bends the picture's timeline to follow the sound. The two start out of step for three reasons: ffmpeg needs 0.1–0.4 s after launch before its first sample is heard, the audio device runs on its own crystal oscillator, and drawing can fall behind when the terminal is busy. An offset of 50 ms is imperceptible; at 150 ms lips are visibly out of sync.

A shared zero point

Both decoders use ffmpeg's `input seek (-ss before -i)`, which decodes forward to the exact requested time. ffmpeg's own `-ss` seeks only to the nearest keyframe, which on a screen recording tested during development was **63 seconds early** (asked for 600 s, it played from 537 s). That is why the audio is decoded by ffmpeg and ffmpeg is used only to play it. Two filters handle streams that start at different times: `aresample=async=1:first_pts=0` pads silence when the audio track begins after the seek point, and `fps=...:start_time=0` repeats the first picture when the video track starts late. In the test film the video begins 0.105 s after the audio.

Reading ffmpeg's clock

With `-stats`, ffmpeg writes a status line to stderr about every 35 ms, for example `1.23 M-A: 0.000 fd= 0 aq= 16KB ...`. The lines end in `\r`, not `\n`, because ffmpeg expects them to overwrite each other on a terminal. `syncToAudio()` drains the non-blocking pipe, finds the **last complete** `\r`-terminated line with `rfind`, and parses its leading number `v`, the seconds of audio played since that ffmpeg started.

Correcting the drift

```
want = audioBase + v - startPos;           // video clock value that matches the sound
drift = secondsSince(t0) - want;           // > 0: the picture is ahead of the sound
if (audioClock && |drift| < 0.25) drift *= 0.25; // damp small jitter
t0 += drift;                               // shift the whole video timeline
audioClock = true;
```

Listing 10. The correction in `syncToAudio()`.

Moving `t0` shifts when every future frame is due, without touching the frame counter. Small errors are corrected a quarter at a time, so the video does not jitter with the noise in ffmpeg's reports. Errors of 250 ms or more (after a stall or a seek) and the very first reading are corrected at once. ffmpeg prints `nan` before playback really starts, and the check `!(v == v)` rejects it, since NaN is the only value not equal to itself.

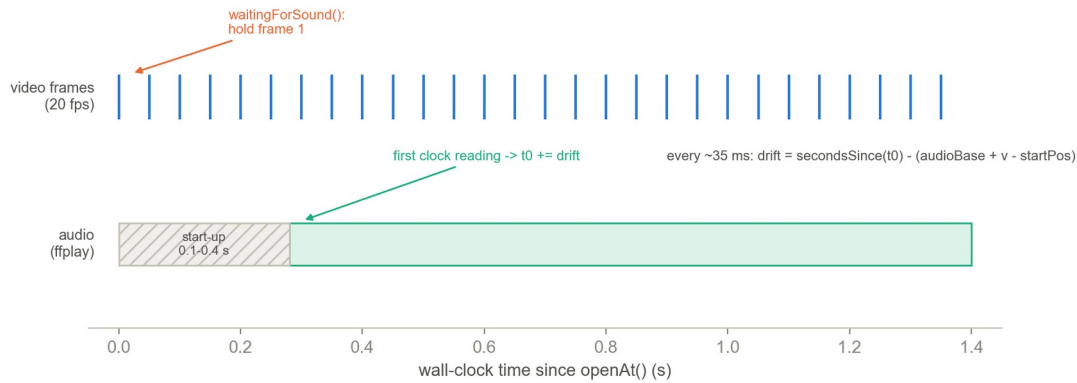


Figure 28. Timeline after `openAt()`: the first frame is held until `ffmpeg` reports a clock

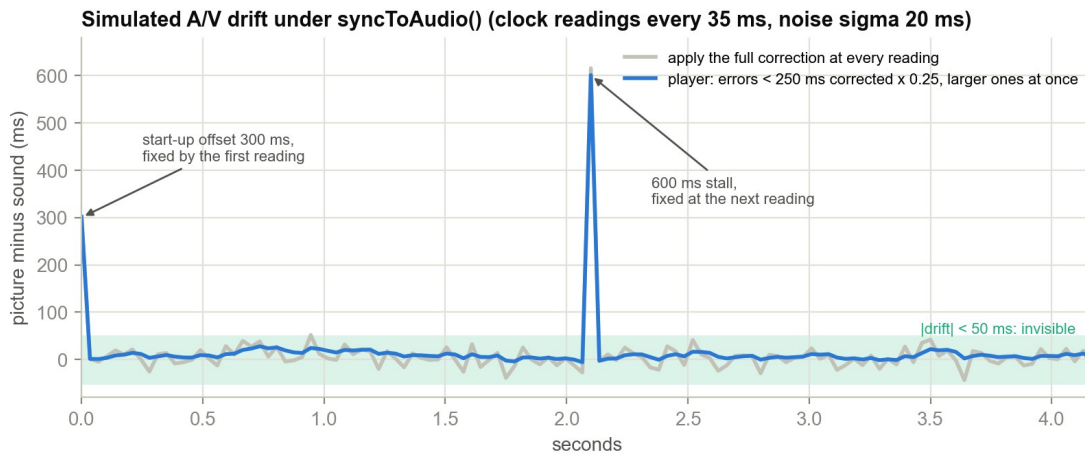


Figure 29. Simulated drift with and without damping, including a start-up offset and a 600 ms stall

Waiting for the sound to start

If the video ran freely during `ffmpeg`'s start-up delay, it would be 0.1–0.4 s ahead when the first sample played and would then jump backwards. `waitingForSound()` is true from the moment the sound is started until `ffmpeg` reports its first real clock value, for at most 2 seconds. During that time the loop shows the first frame (so the screen is not blank) and then holds, polling every 10 ms so that playback starts the instant the sound does.

7.5 Frame Skipping

```
int maxSkip = ramped() ? kRampMaxSkip : int(fps / 2); // 5 in ramp modes
double late = secondsSince(t0) - framesRead / fps;
if (late > 0 && (dropped < maxSkip || late > 0.2) && secondsSince(lastDraw) < 0.5) {
    ++dropped;
    continue; // the frame was read from the pipe, but is not drawn
}
dropped = 0;
render();
```

When the loop falls behind, frames are still *read* (reading is cheap and keeps the decoder in step) but not *drawn*. The condition combines three policies:

- `dropped < maxSkip`: a few skips in a row are normal; up to 5 in the ramp modes.
- `|| late > 0.2`: if the picture is more than 200 ms behind the sound, skip as many frames as it takes to catch up.
- `&& secondsSince(lastDraw) < 0.5`: never go more than half a second without drawing something. A frozen picture looks broken even when the sound is fine.

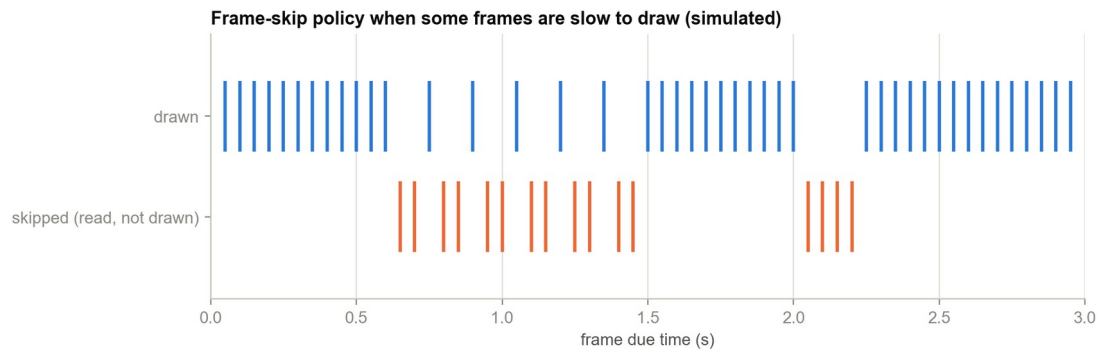


Figure 30. Simulated frame-skip decisions when some frames take longer to draw than the frame period

7.6 Events that Restart Playback

Table 16. Events and how playback responds

Event	Handled by	What happens
Start	run()	openAt(0)
Seek ← / → / ↑ / ↓	seek(± 5 / ± 30)	openAt(position()) $\pm$ delta; if paused, pull and show one frame
Pause	togglePause()	Stop sound, keep decoder and position, redraw status bar
Resume	togglePause()	openAt(position()) restarts picture and sound together
Mute / unmute	handleKey('a')	stopSound() / startSound(position())
Mode change (m)	handleKey('m')	openAt(position()) at the new pixel size; screen cleared
Style (s)	handleKey('s')	Reset drawn[] and tol, clear screen; next frame redraws everything
Charset (c)	handleKey('c')	Switch ramp; cells whose glyph changed are redrawn by the delta test
Window resize	g_resized in run()	If the size really changed: openAt(position())
Font zoom (-, +)	zoomStep()	Terminal resizes → SIGWINCH → same as a window resize

8. Process and Resource Management

During playback the player runs up to five other processes (Figure 31). Managing them correctly, including when things go wrong, took more care than any other part of the program.

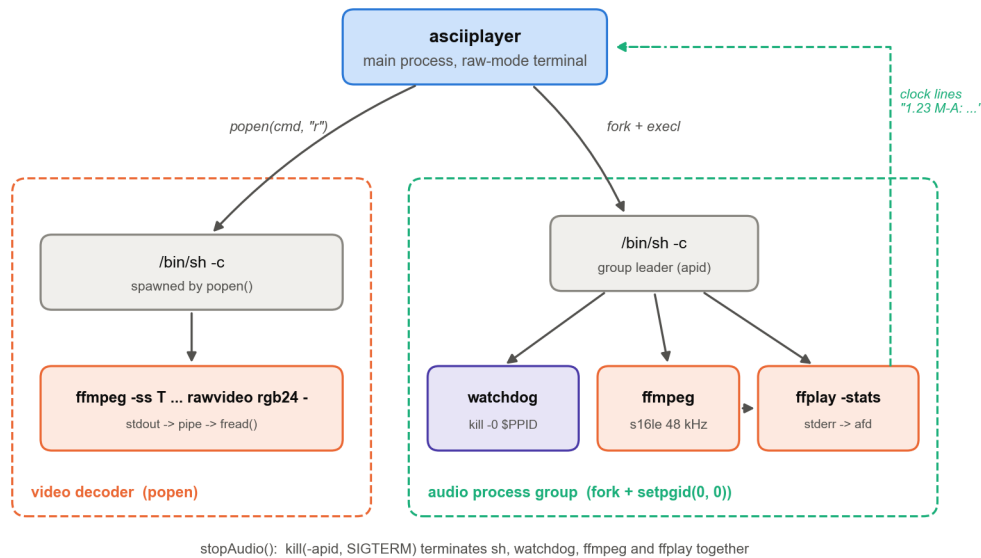


Figure 31. Processes and pipes during playback

8.1 The Video Decoder

The video decoder is started with `popen(cmd, "r")`, which runs `/bin/sh -c cmd` with its stdout connected to a pipe the player reads with `fread`. Each call to `readFrame()` reads exactly `pw × ph × 3` bytes, one frame. A short read means the stream ended. The pipe's descriptor is marked `FD_CLOEXEC` so that it is not inherited by the audio processes started later. `ffmpeg`'s stderr goes to a per-process log file in the temp directory, which is read back for error messages and deleted in the Player's destructor.

8.2 The Audio Process Group

The audio pipeline is three processes: `sh`, `ffmpeg` and `ffplay`, plus the watchdog subshell. Killing only the shell would leave `ffplay` playing. So the child calls `setpgid(0, 0)` to become the leader of a new process group (the parent calls `setpgid(pid, pid)` too, whichever runs first), and `stopAudio()` signals the whole group with a negative PID: `kill(-pid, SIGTERM)`, then reaps the leader with `waitpid`.

8.3 The Watchdog

If the player is killed with `SIGKILL` or crashes, no clean-up code runs, and without protection the sound would keep playing with no window left to stop it. The first command in the audio shell is therefore:

```
(while kill -0 $PPID 2>/dev/null; do sleep 0.5; done; kill -TERM 0) &
```

`kill -0` sends no signal. It only tests whether the parent still exists. When the player disappears, the loop ends and `kill -TERM 0` terminates every process in its own group, the audio pipeline, within half a second.

8.4 Closing Inherited Descriptors

After `fork()` the audio child closes every descriptor from 3 to 1023 before `exec`. The important one is the read end of the video pipe. If `ffplay` inherited it, then when the player closed its own end during a seek, `ffmpeg` would not see a broken pipe, because another reader would still exist. It would block for ever writing to a full pipe, and `pclose()` would hang, freezing the player. This classic fork/exec bug caused the "navigation freezes on files with audio" problem during development (Chapter 12). The fix combines `FD_CLOEXEC` on the video pipe with explicit closing in the child.

8.5 Signals

Table 17. Signal handling

Signal	Default effect	Player behaviour
SIGINT (Ctrl-C)	Terminate	Handler sets <code>g_quit</code> ; loops exit; destructors and <code>atexit</code> restore the terminal
SIGTERM	Terminate	Same as SIGINT
SIGWINCH	Ignored	Handler sets <code>g_resized</code> ; the main loop re-lays out and restarts the decoder
SIGPIPE	Terminate	Ignored, so writing to a closed pipe returns EPIPE instead of killing the player

8.6 Always Restoring the Terminal

`restoreTerminal()` is registered with `std::atexit` and is also called explicitly at the end of `main()`. It is idempotent: it undoes the font zoom, restores the saved `termios`, resets colours, shows the cursor, re-enables line wrap and leaves the alternate screen. Every Player is created in its own block scope, so its destructor (close decoder, stop sound, delete log) runs before control returns to the browser, and before exit.

9. User Interface and Controls

9.1 Playback Controls

Table 18. Playback keys

Key	Action	Notes
Space	Pause / resume	Resume restarts picture and sound in sync
← / →	Seek -5 s / +5 s	Exact: input seek decodes to the frame
↑ / ↓	Seek +30 s / -30 s	Clamped to the file
m	Next render mode	RGB ASCII → MONO ASCII → RGB SHAPES → RGB BLOCKS
c	Next character set	classic → detailed → letters (ramp modes)
s	Next style	rich → lift → exact (ramp modes)
- / +	Smaller / larger text	Terminal.app only; more or fewer cells
a	Mute / unmute	Shown as sound / muted / no-audio
q, Esc, Backspace	Back to the file browser	
Ctrl-C	Quit	Terminal always restored

9.2 Command Line and Environment

Table 19. Command-line usage

Invocation	Effect
<code>./ascioplayer</code>	Open the file browser in the current directory
<code>./ascioplayer FILE</code>	Play FILE immediately; the browser then opens in its directory
<code>./ascioplayer -g [FILE]</code>	Relaunch in a new maximised Ghostty window at 5 pt (macOS)
<code>ASCIIPLAYER_ZOOM=N</code>	Font size in points during playback (default 5); 0 disables zoom
<code>make run</code>	Build, then open the browser

9.3 Screens

The program has three screens: the **file browser** (title, current directory, scrolling list with directories in blue, key help on the last row), the **player** (the video, centred, with the status bar on the last row), and the **message page** for errors. All three are drawn in the alternate screen with synchronized updates, so switching between them never disturbs the user's shell history.

```
ASCII RGB PLAYER  select a video
/Users/prabh/Downloads

> ../
  Movies/
  Screen Recording 2026-05-14 at 16.04.49.mov
  Stranger.Things.Tales.From.85.S01E01 ... .mkv

up/down move  Enter open  left/Backspace parent  ~ home  q quit
```

Listing 11. The file browser (colours omitted).

9.4 Status Bar Behaviour

The status bar (Section 5.8) adapts to the terminal width. On a narrow terminal it drops, in order, the text-size hint, the style hint, the charset hint and the seek details, and finally shows only the time. A clock that no longer fits is truncated rather than wrapped. For live streams with no known duration the progress bar is replaced with blank space.

10. Performance Evaluation

10.1 Test Environment

Table 20. Test environment

Item	Value
Machine	Apple silicon MacBook (M2 class)
Operating system	macOS 26.5.1
Terminals	Terminal.app 2.15 (CPU rendering), Ghostty 1.3.1 (GPU rendering)
FFmpeg	8.1 (Homebrew)
Compiler	Apple clang 17.0.0, -O2
Test film	1280 × 640 H.264, 24 fps, 30 min 55 s, Hindi + English audio
Grid for measurements	320 × 80 cells (25,600 cells), 20 fps, 120 consecutive frames (6 s)

10.2 Bytes per Frame

The number of bytes written per frame is the main factor in smooth playback on a CPU-rendered terminal. It was measured by running the exact rendering and budget logic of `render()` over six seconds of the test film. The results are summarised below and in the figure that follows.

Table 21. Measured average bytes per frame at 320 × 80 cells

Strategy	Average per frame	Write time at 5 MB/s	Fits in 50 ms?
Naive: cursor move + full colour for every cell	1,178 KB	236 ms	No
RGB BLOCKS (full redraw, colour reuse)	772 KB	154 ms	No
RGB ASCII rich, full redraw with colour reuse	597 KB	119 ms	No

Strategy	Average per frame	Write time at 5 MB/s	Fits in 50 ms?
RGB ASCII rich, delta with exact colours	493 KB	99 ms	No
RGB SHAPES (full redraw, colour reuse)	368 KB	74 ms	No
RGB ASCII rich, player (delta + tolerance + budget)	125 KB	25 ms	Yes
RGB ASCII lift, player	117 KB	23 ms	Yes
MONO ASCII, player	21 KB	4 ms	Yes

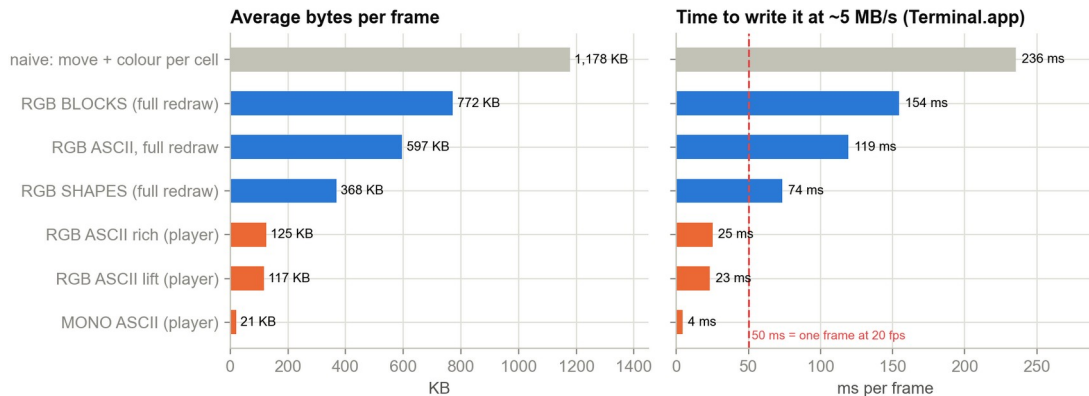


Figure 32. Average bytes per frame for each strategy, and the resulting write time

The player's method reduces the rich RGB ASCII frame by a factor of 4.8 compared with a full redraw, and by 9.4 compared with the naive approach. That brings it from well over a frame period of write time to about half of one. Exact deltas alone save only 17 %, because compression noise changes almost every pixel slightly; the colour tolerance is what makes delta rendering effective. MONO ASCII is 29 times smaller than the full colour redraw because it carries no colour escapes at all. BLOCKS and SHAPES redraw every cell and exceed the budget on Terminal.app at this density. They play smoothly on smaller grids or in Ghostty.

10.3 Terminal CPU Load and Delivered Frame Rate

Even with small frames, Terminal.app has to re-lay-out and paint about 25,000 glyphs twenty times a second on the CPU. Profiling during development showed Terminal.app at around 150 % CPU during full-screen playback, with write stalls of 57–123 ms and visible frame gaps of 100–167 ms. The same playback in Ghostty, which renders text on the GPU, used about 34 % CPU (4.5 times less) and delivered 19.6 of the 20 target frames per second with no stalls (Figure 33). This finding led to the `-g` launcher.

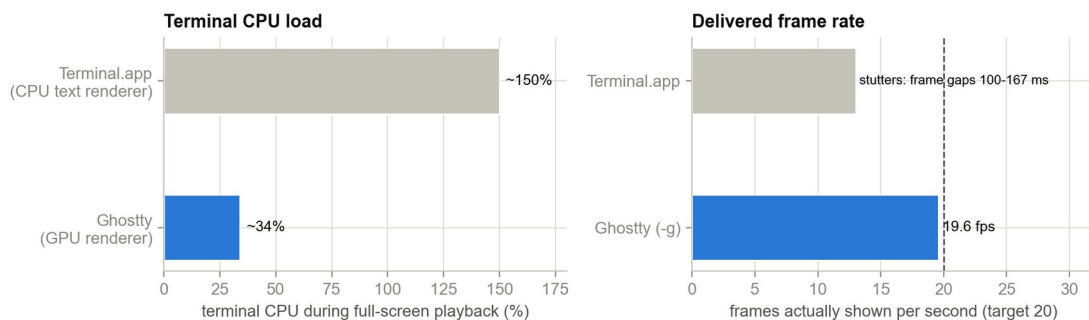


Figure 33. Terminal CPU load and delivered frame rate, Terminal.app versus Ghostty

10.4 Cost of Shape Matching

Scoring all 95 glyphs across 8 sub-cells for every cell would cost about 390 million multiply-adds per second at this grid size and frame rate. The lookup table replaces that with about 24 simple operations per cell. The table itself costs 2.33 million glyph scores once, a few milliseconds, and 24 KB of memory.

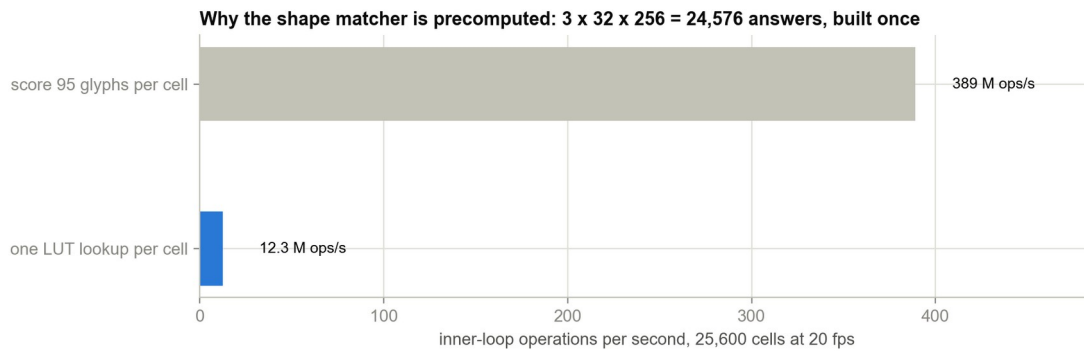


Figure 34. Inner-loop work per second: direct glyph scoring versus the lookup table

10.5 Memory Use

Table 22. Memory use

Buffer	Size (320 × 80 grid)	Lifetime
Frame (ramp modes)	76.8 KB	Per Player; reused every frame
Frame (SHAPES, 644 × 324)	626 KB	Per Player; reallocated on mode change
drawn[] + drawnSaved[]	$2 \times 25,600 \times 4 \text{ B} = 205 \text{ KB}$	Per Player
Output string out	Up to about 0.8 MB capacity	Per Player; capacity reused
Shape lookup table	24 KB	Process; built once
Glyph table	3.4 KB	Static data

Total resident memory for the player itself stays in the low megabytes. The heavy lifting (decoding) happens in the ffmpeg processes.

11. Build, Run and Testing

11.1 Build Process

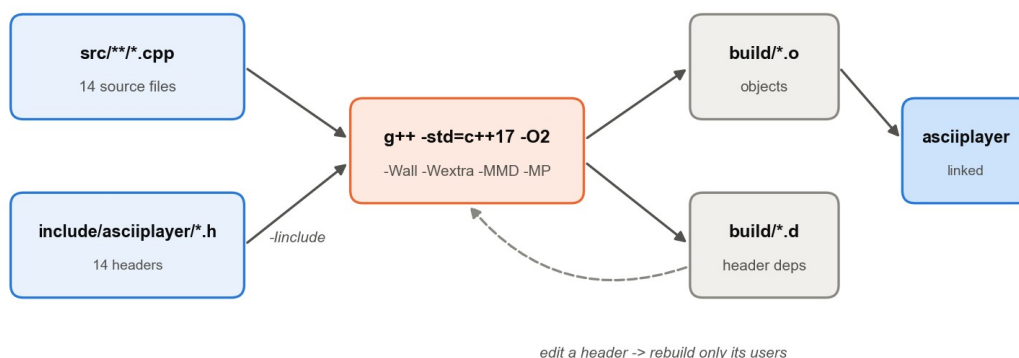


Figure 35. The build process driven by the Makefile

```

$ make          # build ./ascioplayer (only changed files are recompiled)
$ make run      # build, then open the file browser
$ make legacy   # build the original single-file version as ./ascioplayer-legacy
$ make clean    # remove build/ and both binaries

# prerequisites
$ brew install ffmpeg      # macOS
$ sudo apt install ffmpeg  # Debian / Ubuntu

```

Listing 12. Build commands.

The Makefile finds sources with `find src -name '*.cpp'`, maps each to `build/<path>.o`, and includes the generated `.d` files. After editing `modes.h`, for example, only the files that include it are rebuilt.

11.2 Test Media

Table 23. Test media

File	Properties	Why it was useful
Animated series episode (.mkv)	1280 × 640 H.264, 24 fps, 31 min, Hindi (default) + English audio	Long, dark and bright scenes; multiple audio tracks; video starts 0.105 s after audio
Screen recording (.mov)	macOS screen capture, sparse keyframes	Exposed ffmpeg's keyframe-seek error (537 s instead of 600 s)
Phone video (.mp4)	Portrait, short, with audio	Portrait aspect ratio; short-file end handling

11.3 Verification Performed

Table 24. Verification tests

Test	Method	Result
Refactor equivalence	Built the legacy single file and the modular version; played the same file in both and compared the output streams	Functionally identical rendering (legacy binary: 114 frames in 6 s)
Mode cycling	Cycled through all four modes, three charsets and three styles in both binaries	Identical render capability
Exact-style rendering	Checked exact-style output at three terminal sizes against glyphs computed independently from the ramp formula	Zero glyph mismatches
Aspect fit	Checked picture fit and centring across terminal sizes and fonts	Correct fit; no stretching with measured cell aspect
Key handling	Pause, resume, mute, unmute, seek in both directions during playback	All worked; picture and sound resumed in sync
Watchdog	Killed the player with SIGKILL during playback	Audio group terminated within 0.5 s; no orphans
Ghostty playback	Full-screen playback via -g	19.6 fps delivered, no skips or stalls
Error paths	Unreadable and truncated files	Message page with ffmpeg's last errors; back to browser

12. Challenges and Solutions

Most of the design in the earlier chapters came from specific problems met while building and testing the player. This chapter records them, because the reasoning behind each solution is as useful as the code.

Table 25. Problems and solutions

#	Problem observed	Root cause	Solution
1	Picture was blank or showed blocks instead of text	Default mode was the half-block mode	Made RGB ASCII (text glyphs) the default mode
2	Characters too large; picture too coarse	Resolution equals cell count; default fonts give few cells	Font zoom to 5 pt Menlo-Bold during playback via AppleScript; +/- keys
3	Dark scenes almost invisible	Brightness applied twice (glyph density × colour)	Square-root lift; rich style with half-strength background

#	Problem observed	Root cause	Solution
4	Picture stretched or squashed	Assumed a 1:2 cell; fonts vary (0.35–0.6)	Measure cell aspect from pixel size or window geometry
5	Playback freezes and stutters	Frames larger than the terminal can absorb in 50 ms	Delta rendering, colour reuse, adaptive tolerance, adaptive budget, instant retry
6	Sound out of sync after seeking	ffplay -ss seeks to a keyframe (63 s early on one file)	Decode audio with ffmpeg input seek; ffplay only plays
7	Picture ahead of sound at start	ffplay's 0.1–0.4 s start-up delay	Hold the first frame until ffplay reports a clock
8	Slow drift and jitter	Separate clocks; noisy clock reports	Audio as master; damped correction of t0
9	Player froze navigating files with audio	ffplay inherited the video pipe; pclose() hung	FD_CLOEXEC + close all inherited descriptors in the child
10	Sound kept playing after a crash	No clean-up runs on SIGKILL	Process group + watchdog loop
11	Streams starting at different times	Video track begins 0.105 s after audio	fps start_time=0 and aresample first_pts=0 padding
12	Zoom sometimes failed	SIGWINCH interrupted reading osascript output	Retry on EINTR in runCmd()
13	Still stutters in Terminal.app	Terminal.app renders on the CPU (~150 %)	-g launcher for GPU-rendered Ghostty (~34 %)
14	1,435-line single file hard to learn from	Everything in one translation unit	Split into 15 documented modules plus guides and exercises

12.1 Case Study: The Stutter Investigation

The hardest problem was playback that looked fine for a few seconds and then stuttered. Timing instrumentation showed frame gaps of 100–167 ms matching `write()` calls that blocked for 57–123 ms. The player was producing frames on time, but the terminal could not accept them. Two lines of work followed. The first reduced the bytes per frame (Chapter 5) until a typical frame fit comfortably in the budget. The second profiled the terminal itself, which showed Terminal.app's own CPU use at about 150 %: even a perfectly sized frame costs Terminal.app a lot to paint at 25,000 cells. Testing the same build in Ghostty gave smooth 19.6 fps at a quarter of the CPU. The conclusion, that the bottleneck was the terminal and not the player, shaped the `-g` option and the recommendation in the README.

12.2 Case Study: The Refactor

The first working version was a single 1,435-line file. It was split into modules along the lines of Figure 3. Each header documents the module's purpose, dependencies and design decisions, and a reading order was written in `docs/00-START-HERE.md`. To prove nothing changed, the legacy file is still built by `make legacy`, and both binaries were run against the same inputs and compared (Table 24). The split also made visible which state is genuinely global (Section 3.5) and which belongs to the Player.

13. Limitations and Future Work

13.1 Current Limitations

- **Terminal throughput** limits the full-colour modes on CPU-rendered terminals. BLOCKS and SHAPES at full-screen 5 pt density exceed Terminal.app's budget.
- **Font zoom** is implemented only for macOS Terminal.app. Elsewhere the user sets the font size by hand.

- **POSIX only.** The code depends on `termios`, `fork` and `process` groups. Windows consoles are not supported (WSL works).
- **Ramp modes run at a fixed 20 fps.** High-frame-rate sources are resampled down.
- **Single-byte ramps.** Ramp strings are indexed as bytes, so multi-byte UTF-8 shading characters (such as ) cannot be used in a ramp without changes.
- **No subtitles, playlists or network streams**, beyond what `ffmpeg` accepts as a path.
- **The shape matcher uses a fixed 1:2 cell** for its sub-grid rather than the measured cell aspect.

13.2 Future Work

1. Apply delta rendering and the byte budget to the SHAPES and BLOCKS modes as well.
2. Use the measured cell aspect for the SHAPES layout, and measure glyph ink for the actual playback font at run time.
3. Support UTF-8 ramps and Unicode block and Braille characters for finer sub-cell detail (Braille gives 2 × 4 dots per cell).
4. Render subtitles by asking `ffmpeg` to burn them into the frames (`subtitles=` filter).
5. Add font-size control for other scriptable terminals (`Ghostty`, `kitty`, `WezTerm` expose their own control protocols).
6. Show both clocks on the progress bar to visualise sync correction (listed in `docs/05-exercises.md`).
7. Add a simple on-screen FPS and bytes-per-frame overlay for diagnostics.

14. Conclusion

The project shows that a plain text terminal can serve as a usable video display. The player plays real films with sound, in colour, in real time, and responds to controls like a normal media player. It stays small (about 2,000 lines of C++17 with no libraries beyond the standard library and POSIX) by leaving decoding to `FFmpeg` and concentrating on the problems specific to the medium.

On the conversion side, the report described four techniques and the reasons behind each refinement. Perceptual luminance weighting makes tones correct. The square-root lift keeps dark scenes visible by splitting brightness between glyph density and colour. The half-block mode delivers the most faithful image the grid can hold. The shape matcher uses measured glyph ink, a two-part level and shape score, contrast planes and a 24,576-entry precomputed table so that characters follow the edges in the picture at the cost of one array read per cell.

On the rendering side, the key result is that bandwidth, not computation, limits terminal video. Delta rendering, colour reuse, quantisation, an adaptive colour tolerance and a feedback-controlled byte budget together reduce a typical frame from about 597 KB to 125 KB without visible loss, which brings 20 fps within reach of a CPU-rendered terminal. A GPU-rendered terminal removes the remaining bottleneck. Treating the audio clock as the master, with damped correction and a shared exact seek point, keeps picture and sound together.

Finally, the code is organised as a set of small, documented modules with a reading guide and exercises, so the project also works as a study of terminal programming, process management, real-time timing and image processing in C++.

References

1. FFmpeg Developers. *FFmpeg documentation: ffmpeg, ffprobe, ffplay, filters (scale, fps, aresample)*. <https://ffmpeg.org/documentation.html>
2. International Telecommunication Union. *Recommendation ITU-R BT.601: Studio encoding parameters of digital television*.

3. ECMA International. *ECMA-48: Control Functions for Coded Character Sets*.
4. Thomas E. Dickey. *XTerm Control Sequences*. <https://invisible-island.net/xterm/ctlseqs/ctlseqs.html>
5. IEEE and The Open Group. *POSIX.1-2017 (IEEE Std 1003.1): terminos, poll, fork, exec, setpgid, sigaction*.
6. ISO/IEC 14882:2017. *Programming Languages: C++ (C++17)*.
7. cppreference.com. C++ standard library reference: `std::chrono::steady_clock`, `std::filesystem`, `std::to_chars`. <https://en.cppreference.com>
8. Contour Terminal / community specification. *Synchronized Output (DEC mode 2026)*.
9. Ghostty. *Ghostty terminal emulator documentation*. <https://ghostty.org>
10. Apple Inc. *Terminal Scripting (AppleScript dictionary for Terminal.app)*.
11. Unicode Consortium. *Block Elements, U+2580 UPPER HALF BLOCK*.
12. asciiplayer source code and documentation: README.md and docs/00–05.

Appendix A. Quick Reference

Table 26. Complete key reference

Context	Key	Action
Browser	↑ ↓ PgUp PgDn	Move selection
Browser	Enter / →	Open directory or play file
Browser	← / Backspace	Parent directory
Browser	~	Home directory
Browser	q / Esc	Quit
Player	Space	Pause / resume
Player	← →	Seek 5 s
Player	↑ ↓	Seek 30 s
Player	m	Render mode
Player	c	Character set
Player	s	Style
Player	- +	Text size (Terminal.app)
Player	a	Mute
Player	q / Esc / Backspace	Back to browser
Anywhere	Ctrl-C	Quit, restoring the terminal

Table 27. Tuning constants

Constant	Value	Meaning
kRampFps	20	Frame rate in ramp modes
kRampMaxSkip	5	Max consecutive skipped frames (ramp modes)
kQuant	4	Colour quantisation step
kFrameBudgetStart	180,000 B	Initial frame budget
kFrameBudgetMin / Max	40,000 / 400,000 B	Budget bounds
kMaxToL	64	Maximum colour tolerance
kSubX × kSubY	2 × 4	Shape sub-grid
kLevels × kPlanes	32 × 3	Shape LUT dimensions (× 256 signatures)

Constant	Value	Meaning
Plane thresholds	14 / 40	Average-spread limits for planes 0 / 1 / 2
Default zoom	5 pt Menlo-Bold	ASCIIPLAYER_ZOOM overrides
ESC timeout	25 ms	Escape-sequence byte timeout in readKey()
Sound wait	2 s	Max hold for ffmpeg's first clock
Sync damping	× 0.25 below 250 ms	Small drift correction factor
Catch-up threshold	0.2 s	Lateness that allows unlimited skipping

Appendix B. Glyph Ink Table

All 95 glyphs of `src/render/glyphs.cpp` with their mean ink, normalised so the densest glyph **N** has mean 1.0, read column by column in increasing order of ink (sp = space). Read in this order, the table is a 95-step brightness ramp. Each glyph's ink in the eight 2×4 sub-cells (Figure 14) is stored alongside the mean in the source file.

Table 28. Measured mean ink of the 95 glyphs (Menlo)

Glyph	Mean	Glyph	Mean	Glyph	Mean	Glyph	Mean	Glyph	Mean	Glyph	Mean
sp	0.000	+	0.371	j	0.475	y	0.571	S	0.664	K	0.768
`	0.075	(	0.375	7	0.482	I	0.583	X	0.666	O	0.784
'	0.158	)	0.376	L	0.482	F	0.590	\$	0.698	H	0.792
.	0.161	<	0.392	f	0.484	o	0.593	P	0.701	g	0.794
-	0.204	>	0.392	x	0.486	2	0.607	m	0.717	#	0.803
,	0.219		0.398	*	0.490	e	0.617	G	0.718	D	0.813
_	0.234	=	0.405	s	0.499	w	0.624	p	0.723	8	0.833
:	0.256	?	0.414	{	0.507	V	0.628	q	0.723	R	0.836
^	0.286	l	0.422	}	0.507	h	0.633	d	0.724	Q	0.841
~	0.288	c	0.430	Y	0.517	3	0.635	b	0.725	0	0.894
!	0.288	i	0.446	T	0.523	k	0.636	A	0.725	W	0.895
"	0.309	[	0.458	J	0.524	a	0.639	U	0.732	M	0.912
;	0.334	v	0.458	1	0.538	%	0.654	E	0.732	B	0.914
r	0.356	]	0.458	u	0.542	Z	0.656	&	0.765	@	0.934
\	0.363	t	0.470	n	0.543	4	0.659	6	0.768	N	1.000
/	0.363	z	0.473	C	0.547	5	0.662	9	0.768		

Appendix C. ANSI Escape Sequences Used

Table 29. Escape sequences

Sequence	Name	Used for
ESC[?1049h / ESC[?1049l	Alternate screen on / off	Keep the user's scrollbar intact
ESC[?25l / ESC[?25h	Hide / show cursor	No blinking cursor over the film
ESC[?7l / ESC[?7h	Auto-wrap off / on	A glyph in the last column must not scroll the screen

Sequence	Name	Used for
ESC[2J	Erase display	Mode change, resize, style change
ESC[2K	Erase line	Clear the status row
ESC[H	Cursor home	Browser redraw
ESC[row;colH	Cursor position (1-based)	Start of every row / changed-cell run
ESC[38;2;R;G;Bm	24-bit foreground	Glyph colour
ESC[48;2;R;G;Bm	24-bit background	Rich-style fill; lower half-block pixel; status bar
ESC[38;2;...;48;2;...m	Combined fg + bg	Rich style: both colours in one sequence
ESC[0m	Reset attributes	End of every row
ESC[7m	Reverse video	Browser selection highlight
ESC[1m	Bold	Browser title
ESC[?2026h / ESC[?2026l	Begin / end synchronized update	Show each frame atomically (no tearing)

Keyboard input sequences decoded

Table 30. Input sequences

Key	Bytes
Up / Down / Right / Left	ESC [A / B / C / D (or ESC O A ...)
Page Up / Page Down	ESC [5 ~ / ESC [6 ~
Enter	CR (0x0D) or LF (0x0A)
Backspace	DEL (0x7F) or BS (0x08)
Escape	ESC alone (no byte within 25 ms)

Appendix D. Glossary

Table 31. Glossary

Term	Meaning
ANSI escape sequence	Bytes beginning with ESC that control a terminal instead of printing
Alternate screen	A second screen buffer; programs like vim use it so the shell's history is restored on exit
Canonical / raw mode	Line-buffered, echoing terminal input versus immediate, unechoed key input
Cell	One character position in the terminal grid
Cell aspect	Width ÷ height of a cell, about 0.5 for most monospace fonts
Delta rendering	Redrawing only cells whose content changed since the last frame
Glyph	The visual shape of a character
Input seek	ffmpeg -ss before -i: decode forward to the exact requested time
Keyframe	A video frame decodable on its own; other frames depend on it
Luminance (luma)	Perceived brightness computed from weighted R, G, B
Lookup table (LUT)	Precomputed answers indexed by the inputs, replacing computation with a memory read
Master clock	The clock the other stream is adjusted to follow; here, the audio

Term	Meaning
PCM (s16le)	Uncompressed audio as signed 16-bit little-endian samples
Process group	A set of processes that can be signalled together with kill(-pgid)
Ramp	Characters ordered from least to most ink
RGB24	Raw pixel format: one byte each of red, green and blue per pixel
SGR	Select Graphic Rendition: the ESC[...m sequence that sets colours and attributes
Signature	8-bit pattern of which sub-pixels are brighter than a cell's mean
Steady clock	A monotonic clock unaffected by changes to the system time
Synchronized update	Terminal mode 2026: buffer output and display it in one step
termios	The POSIX structure and API that configures a terminal's input processing
True colour	24-bit colour, 16.7 million values, per character